OPERATORS: **#08/07/2025**

1.Arithmetic operators:

+:

- Used with minimum of 2 numbers perform ➔ addition
- Used with single number to represent a positive number

```
print(5+3+2) #addition
print(76.9+86.90) #addition
print(2+5+8+9+23.6+0) #addition
print(+27) #positive number
print(+78.90) #positive number
```

-:

- Used with minimum of 2 numbers perform ➔subtraction
- Used with single number to represent a negative number

```
print(56-89-76-20) #subtraction
print(48-87) #subtraction
print(102.3 - 67.90 - 76.50) #subtraction
print(-23.89) #negative number
print(-2) #negative number
```

*:

➔ Used with compulsory, minimum of 2 numbers perform ➔Multiplication

```
print(10*6*4*3*20) #multiplication
print(12.46*78.34) #multiplication
print(2*56.7) #multiplication
print(*8) #error
```

**** : [EXPONENTIATION]**

- ➔ Return the exponent value.
- ➔ 2^3, where 2 is base and 3 is power. Answer will be 2*2*2=8
- ➔ SYNTAX :

  base**power

```
print(2**3)
print(3**2)
print(2.36**3.25)
```

Division Operator: **#09/07/2025**

a) **Floor div[//]**

- ➔ Return “non decimal quotient”

Example : print(10//2) #5

Print(22//7)#3

b) **Float div[/]**

- ➔ Return “decimal quotient”

Example: print(10/2) #5.0

Print(22/7) #3.1428

c) **Modulous[%]**

- ➔ Returns “remainder”
- ➔ If left hand side no is lesser than right hand side no then the output will be left hand side no.

Example: print(10/2) #0

Print(22/7) #1

Print(2%22) #2

Print(5%888) #5

2.Comparsion or Relational operator

➔ return Boolean value

➔Ture[1], False[0], None[] are special type of pre-defined keywords in python (they will literally hold the values)

➔other keywords holds the meaning

```
print(10<15) #True
print((2*10)>16) #True
print((10*10)<=(10**2)) #True
print(True>=1)     #True       #True,False,None(this will litraly hold the values) are special keywords
print((2**False)<=(2**True)) #True
print((True+True)!=(False-False)) #True
print("a"!=65)    #True        # ==, != they are relation operator
print("a">66)     #Error         # > and < they are comparison operator, we don't need to compare a strin
```

3.Logical Operator #10/07/2025

➔ return Boolean value or anything that represent Boolean value

NOTE: BOOLEAN CONDITION

- Direct Boolean value
- Anything that represent Boolean value
- An expression that return a Boolean value

a) And

➔ Whenever we need to compare and check multiple Boolean condition

➔Any number other than Zero is considered to be True

C1	C2	O/P
T	T	T
T	F	F
F	-	F

```
print(((2*10)!=(10+10)) and True and (True!=0)) #TRUE
print(2 and True)  #True
print(-2 and True)   #True
print(0 and True)   #0
```

b) OR

C1	C2	O/P
T	-	T
F	T	T
F	F	F

```
print(2 or 0 or -1)  #2
print((True**False) or (True**True) or (False and True)) #1
print((False and True or True)or(10>5)) #True
print((False or True and False)or(10>5)) #True
```

c) NOT

C1	O/P
T	F
F	T

```
print(not((10+3)and (10<15) and(8)))   #False
print(not(not((True and False) or(1 and 2) or(False)))) #True
```

4.Bitwise Operator

a) **Bitwise and (&)**
b) **Bitwise or (|)**
c) **Bitwise not (!)**
d) **Bitwise xor (^)**
e) **Bitwise left shit (<<)**
f) **Bitwise right shit (>>)**

5.Assignment Operator [=] **#11/07/2025**

➔does not return any value

➔To update and store any value to a named memory location

➔Syn:

a) **Initialization(Storing):**
   Var_name=value
   Eg: a=10
b) **Copying:**
   Initialized var
   Var_name= initialized var
   Eg: b=200 / c=b

```
a=10      #variable= a named memory location
b=(2*5)
c=a
d=c
print(a) #10
print(b) #10
print(c) #10
print(d) #10
d=15
print(d) #15
d=100
print(d) #100
d=25
print(d) #25
e=10,20,30
print(e) #(10,20,30)
print(type(e)) #<class 'tuple'>  Type is a in-built function which will return data type of variable
print(x) #error
z=r+s #error
```

Variable :

- It is named memory location which holds the single valued data
- It is container just to hold the value
- It always stores , the latest updated value
- If we try to store same value into multiple variables then PVM will provide the same memory to all the variables.
- If we try to store multiple values together into a same variable then PVM will convert into a TUPLE
- It is compulsory to initialize a variable before utilizing it

  NOTE:

  type(): it is a pre-defined function that returns the data type of a initialized variable or a value passed

```
a=10
b=20
sum=a+b
a=a+b
print(a) #30
print(b)  #20
print(sum)  #30
a=10
b=20
a=a+b
a+=b
print(a)
print(b)
x=5
y=2
x=x**y
x**=y
print(x)
```

(Types or Compound Statements) of Assignment Operators **#12/07/2025**

-Combination of arithmetic or bitwise with assignment operators

a) **Arith + Assign:**
+ =, - =, * =, //=, /=, %=, **=

b) **Bitwise + Assign:**
&=, |=, ^=, <<=, >>=

```
a=3
a-=2
print(a)        #1

x=10
y=3
x-=x
print(x)        #0
print(y)

a=10
y=3
#a*=z
print(a)
print(y)
#print(z)      #error

a=5
#a~=a       #~ : negation   #unvalid
print(a)     #error

a=~a     #valid
print(a)

x=5
y=3
y=x-y #we cannot convert this compound statment
print(x)
print(y)
```

RULES:

- It is compulsory to use initialized variables, if any in compound statement's
- Constants are allowed in compound statement's expression
- Minimum of 2 operands are compulsory to be converted as compound statements
- Whenever the same operand is used on the RHS(for operation) and LHS for updation
- It is a good practice to locate the operand of left of the RHS operation and the same operand on the LHS for updation, if not it cannot be converted to compound staments

6.Identity Operator

➔ returns Boolean value

➔Compare and check whether the id's of the given object's are same or not

➔id : unique integer representation assigned by the PVM based on the memory provided

➔id() : pre-defined function that returns the id generated for a memory

```
a=5
print(id(a))    #136651035952
s1="abc"
print(id(s1))    #206991234128

a=100
b=100
c=100
d=a
print(a==b==c==d) #same memory is shared
print(a is d)    #True
print(b is c)    #True
print(a is c)    #True

x=10
print(x) #10
y=x
print(y) #10
print(x is y) #True

x=10
print(x)    #10
print(id(x))    #unique code
x=20
print(x)    #20
print(id(x))    #unique code
x=30
print(x)    #30
print(id(x))     #uniqye code
```

NOTE :

Iterable Object: The objects that can hold multiple individual elements into the same shared memory location

Eg : list, set, tuple, string, dictionary,….

L1=[10,-2,25.6,'c']

Print(l1[2]) #25.6

Print(l1[0]) #10

7.Membership operator: [in, not in] **#14/07/2025**

➔Returns boolean value

➔it checks whether the specified element is available in the group of elements or iterable objects or not

```
s1="abbc123"
print("a" in s1)      #True
print("bbc" in s1)     #True
print("12" in s1)     #True
#print(23 in s1)      #error

a=10    #single value data
#print(1 in a)    #error

a="10"
print("1" in a)    #True

a=""
print("ab" in a)      #False
```

CONTROL FLOW STATEMENTS :

➔The statements that controls the execution flow of a program

➔3 Types:

a) Conditional Statements:
   a) Simple -if
   b) If-else
   c) elif/if – elif / elif ladder

Note : Combination of these above statement's are "nested if conditions"

b) Looping Statements:
   a) For loop
      I. for with range()
         i. increment loop
         ii. decrement loop
      II. for without range()
   b) while loop
      i. increment
      ii. decrement

1.Conditional Statements

a) if statements:

```
age=int(input("enter age:"))
if age>=18:
    print("you can vote")
print("you are a citizen")
```

b) if-else statements:
[NOTE: Divisibility / Factor → %]

```
num=int(input("enter the no : "))
if num % 2== 0:
    print(f"{num} is a even number")   #f means formatted printing
else:
    print(f"{num} is a odd number")
print("program executed")
```

c) elif ladder: **#15/07/2025**

➔It will be used when in a given scenario where will be multiple condition checked and for each condition there will be specific condition

➔Syntax:

```
If bool_cond1:
    #logic1
Elif bool_cond2:
    #logic2
Elif bool_cond3:
    #logic3……
else:
    #alternative logic
#remaining statements
```

```
if n > 0 and n % 2 == 0:
    print(f"{n} is positive even number")
elif n > 0 and n % 2 != 0:
    print(f"{n}is positive odd number")
elif n<0 and n%2==0:
    print(f"{n} is negative even num")
elif n<0 and n%2!=0:
    print(f"{n} is negative odd num")
else:
    print(f"{n} is neutral")
```

TASK :

WAP to print the following statement's accordingly;

a) if the number is positive but even ➔ positive even num
b) if the number is negative but even➔negative even num
c) if the number is positive but odd➔positive odd num
d) if the number is negative but odd➔negative odd num

Note: do the task using nested "if" condition

```
n=int(input("enter a number: "))
if(n>0):
    if(n%2==0):
        print(f"{n} positive even number")
    else:
        print(f"{n} positive odd number")

elif(n<0):
    if(n%2==0):
        print(f"{n} negative even number")
    else:
        print(f"{n} negative odd number")
else:
    print(f"{n} number is neutral")
```

2.Looping Statements **#16/07/2025**

➔we use it to repeat the logic for certain number of times

1. **For loop:**
    a) **For with range():**

range():

➔pre defined function to set up a range of values

Syn:

range(start, stop/end, step)

start, stop, step ➔only integers

start ➔from where the range should begin

default start = 0

stop/end ➔till where the range should get executed

it is compulsory to pass

step ➔uniform difference b/w the current and the next value

default step = +1

```
print(range(1,4+1,+1))  #range(1,5)
print(range(10))  #range(0,10)
#print(range("10"))  #error
print(range())   #error
```

[NOTE: As range() is setting up the sequence of value's internally Therefore, range() requires "for" loop to access each individual sequence value]

General Syntax for loop with range():

```
for var_name in range(start, stop, step):
        #logic
#rem statements
```

Types of "**for loop**" with **range()**:

1. **Increment loop**

   Syntax:

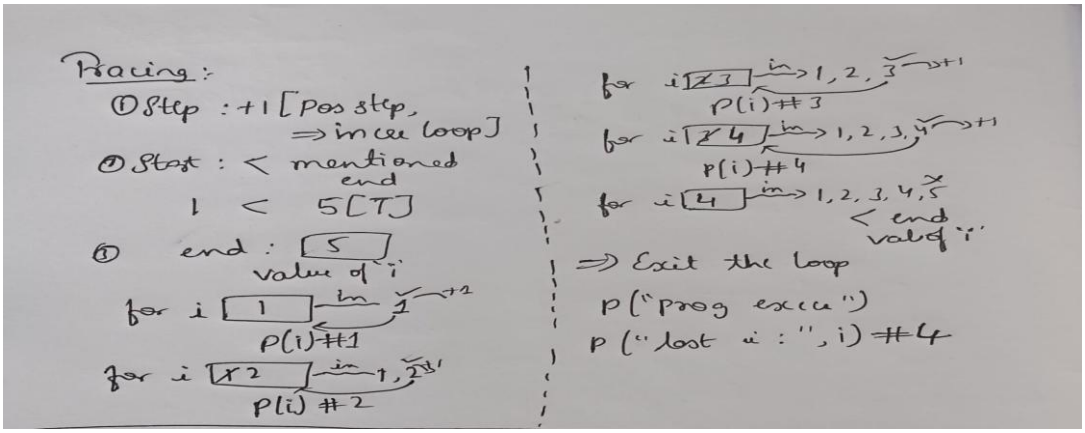
```
for var_name in range(start, (actual end+1), positive step):
        #logic
#rem statements
```

   [NOTE : Tracing ➔ step by step execution of code during run time]

   Rules of Increment Loop:

   i. Positive step
   ii. Start should be less than mentioned end
   iii. If we need to include the actual end also then, actual end+1

```
n=int(input("enter a num : "))
for i in range(1,n,+1):
    print(i, end=" ")
print("\n pgm executed")
print("last i:", i)
```



2. **Decrement loop**

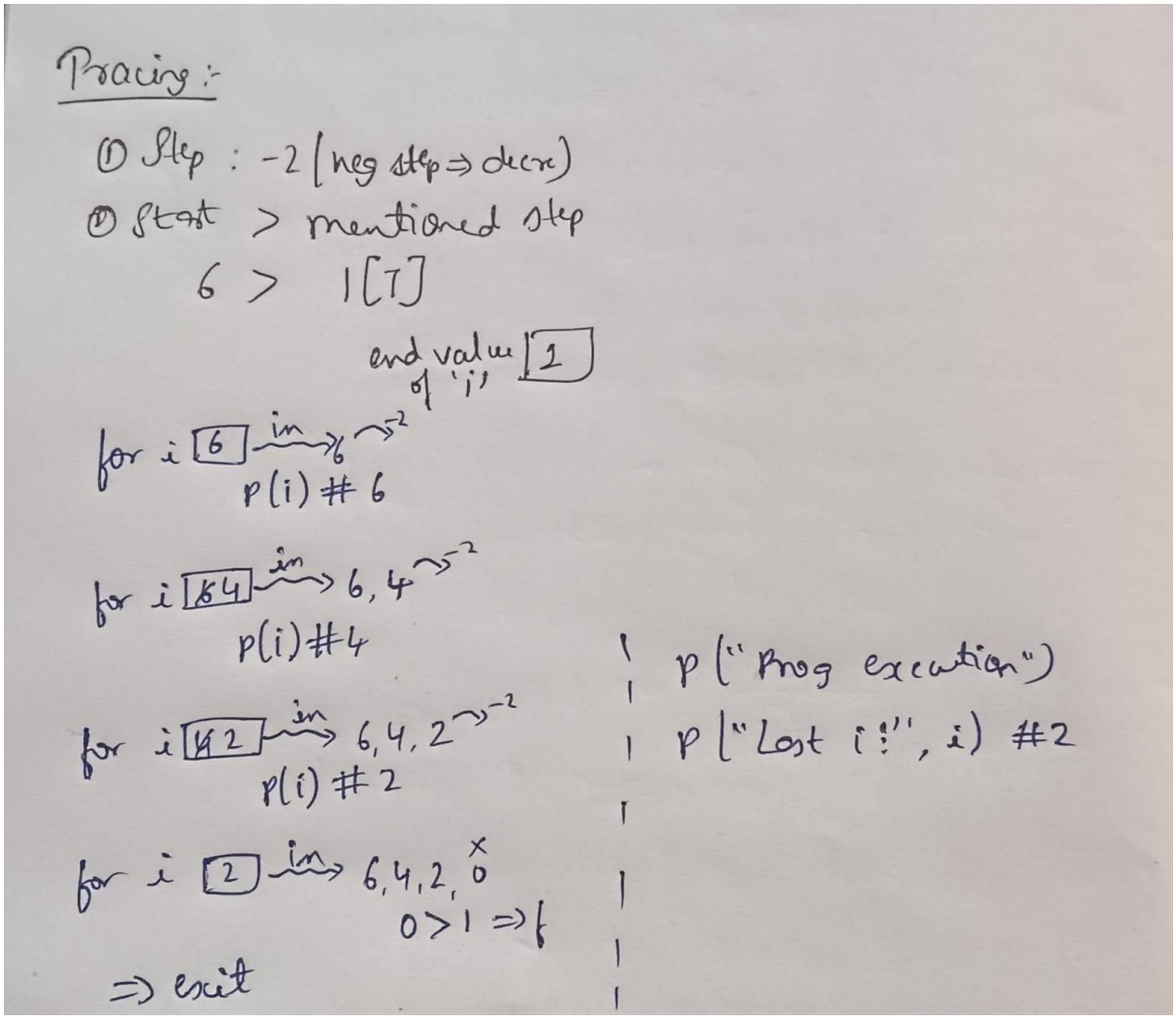
Syntax:

```
for var_name in range(start, actual end-1, negative step):
    #logic
#remaning statements
```

Rules of Decrement Loop:

i. Negative step
ii. Start should be greater than actual end
iii. If we need to include the actual end also then, actual end-1

```
n=int(input("enter a num: "))
for i in range(n,2-1,-2):
    print(i,end=" ")
print("\npgm executed")
print("last i:",i)
```



For loop without range(): **#21/07/2025**

Syntax:

For var_name in (iterable objection)/(group of element):

#logic

#remaning statement

Rules:

- It will work only on (iterable objectives)/(group of element):
- Looping var will point to the element directly
- The cursor will move only in forward direction
- Throws error where the I/P is “single valued data”
- It even works on empty data structure
  Eg: “ ”,[],(),……
- Values of different data types can be grouped and given as I/P

```
for i in(10,20,30,-3):
    print(i)                    #10 20 30 -3

for i in "sharan":
    print(i,end=" ")            #s h a r a n

for i in "10067":
    print(i, end="")            #10067

for i in "":
    print(i, end="")

for i in ():
    print(i,end="")

for i in "jj shkail kankl":
    print(i, end="")             #jj shkail kankl

for i in 10067:
    print(i, end="")            #error
```

Difference between “For with range()” and “for without range()”

- In for with range(), where we do not required lot of pre-defined function
- In for without range(), at any cost we need pre-defined function

```
l1=[1,2,3]     #with range
for i in range(len(l1)):
    print(i, end="")   #012

print("\n=======================")
for i in l1:    #without range
    print(i,end="")    #123

print("\n=========================")
l2=[1,2,3]
for i in range(len(l2)):   #with range
    print("ele :",l2[i],"index:",i)   # ele: 1 index:0 \n ele:2 index:1   \n ele:3 index:2


print("\n=========================")
for i in l2:   #without range
    print("ele :",i,"index:",l2.index(i))  # ele: 1 index:0 \n ele:2 index:1   \n ele:3 index:2
```

2.WHILE LOOP (When we know the start value and don't know the end value)

Syntax:

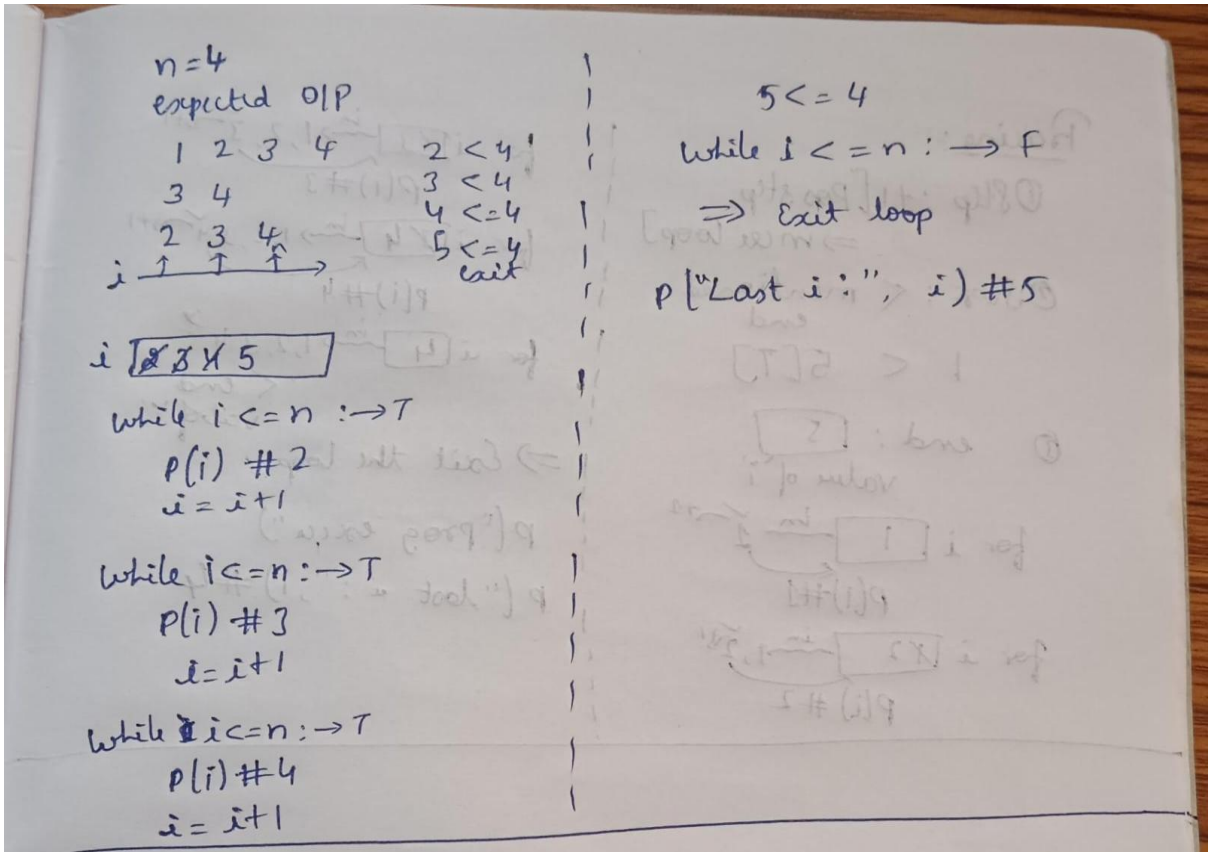
```
#initialize the variable to be used in bool_condition
While bool_condition:
    #logic
    #updation of looping variables
#remaning statements
```

Types of While Loop

a) Increment
b) Decrement

WAL to print n natural numbers: **#22/07/2025**

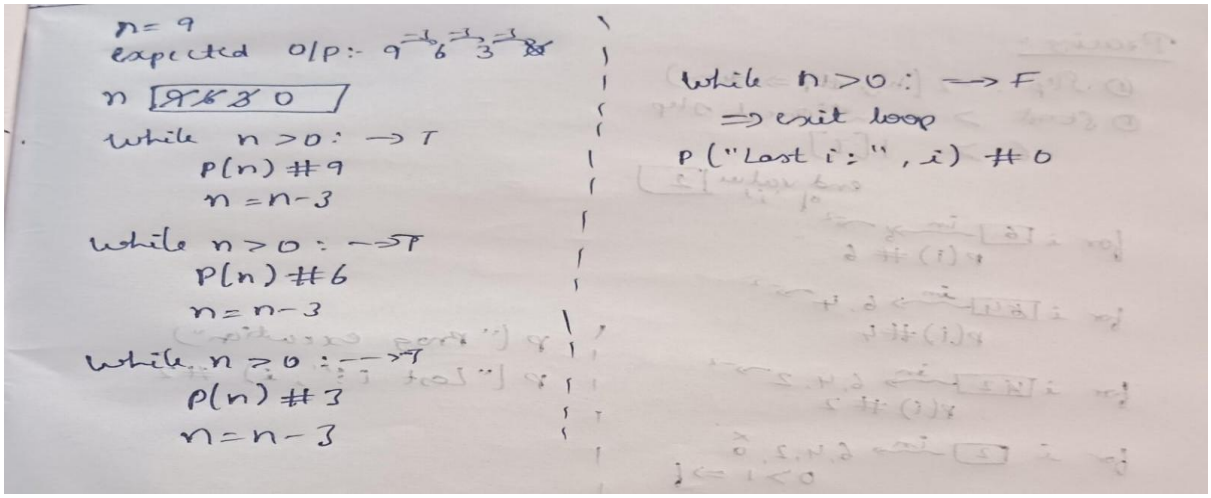
```
n=int(input("enter the no : "))    #7
i=0
while i<=n:              # i:looping_variable
    print(i, end=" ")
    i += 1                  # print 0 to 7

print("\nlast i:",i)   #8
```



WAL to print n natural numbers in decrement order with a difference of 3

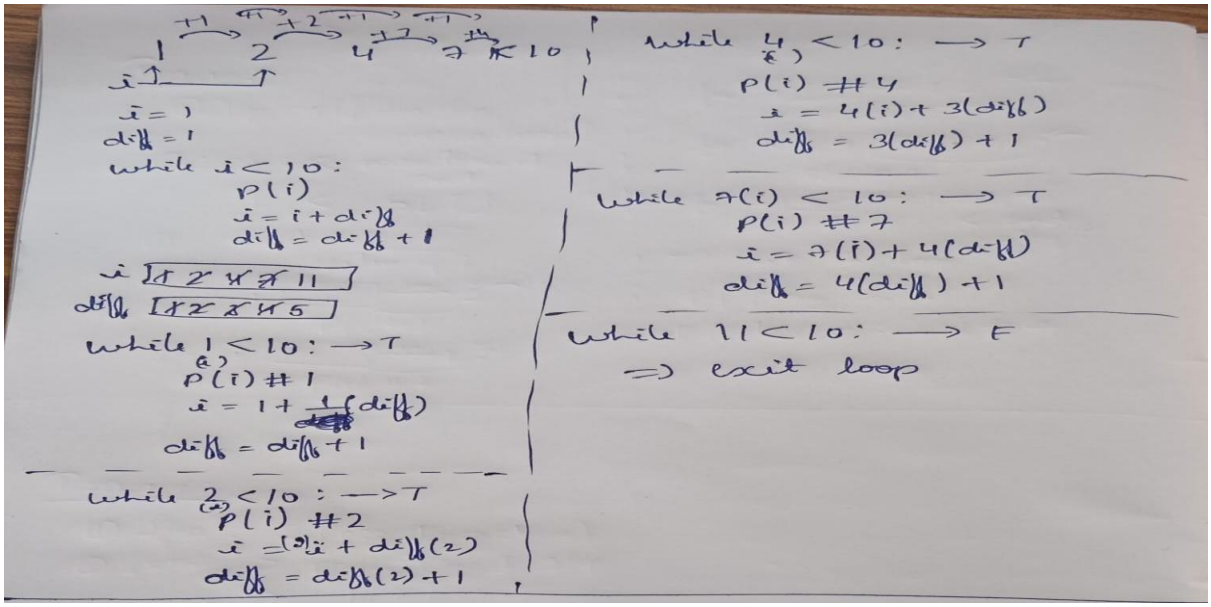
(In this program we don't need use the looping extra variable)

```
n=int(input("enter the no : "))    #9
while n>0:
    print(n, end=" ")    #9 6 3
    n -= 3
print("\nlast n:",n)  #0
```



WAL to print numbers difference between the value should increase sequentially

```
n=int(input("enter a no : "))    #1
c=1
while n<10:
    print(n,end=" ")    #1 2 4 7
    n+=c
    c+=1
```



WAL to get the I/P from user and keep printing until and unless the user is not entering 5.once the user enters 5 stop asking for I/P

```
while True:
    n=int(input("enter a num:"))
    if n==5:
     break
    print(n)
print("program executing")
```

[NOTE: break➔It is keyword to exit the current executing loop]
[NOTE: continue➔it is keyword to skip a cycle in the current executing loop without exiting the loop]

FUNCTIONS **#23/07/2025**

➔It is block of code to perform a specific task

Usage of Function

➔Once the code is return within the function it can be reused just by calling the function anywhere for any number of times
➔Optimization of code

Syntax:

```
def func_name(parameters):
        #logic
        Return val(if any)

func_name(arguments)
        or
var_name=func_name(arguments) ➔ (if any return value)
```

LOOPS VS FUNCTIONS

LOOPS	FUNCTIONS
Loops is used to repeat a logic at the same place	It is used to repeat a logic at different places

Parameters VS Arguments

Parameters	Arguments
The I/P variable required by a function to perform the functionality They are dynamic local variable	The values passed to parameters of function during the **function called**

```
def sub(a,b):
    sub=a-b
    return sub
    print("Hello")
res=sub(10,3)
print(res)      #7
print(res+3)    #10
print(res*10)   #70
```

[NOTE: return

➔It is a keyword, it return value/values from called function back to the function called

➔It return the execution flow of the program from called function back to function called

➔It is last executable line in the function

➔No other line included with the function after the execution of the function]

WAP to find the number is even or odd using Customized function

[NOTE : flag is standard variable name to hold the Boolean value]

```
def isevenodd(n):
    if n % 2==0:
        return True  #even
    else:
        return False  #odd

n=int(input("enter a num: "))    #15
flag= isevenodd(n)  #flag is standard variable name to hold the boolean value
if flag:
    print(f"{n} is even")
else:
    print(f"{n} is odd")     #15 is odd
```

```
def isevenodd(n):
    n % 2==0

n=int(input("enter a num: "))    #5
flag= isevenodd(n)  #flag is standard variable name to hold the boolean value
if flag:
    print(f"{n} is even")
else:
    print(f"{n} is odd")     #5 is odd
```

#24/07/2025

WAP to print all the even numbers and odd numbers separately of a user defined range

```
def isevenodd(n):
    return n%2==0
sr=int(input("enter start range : "))
er=int(input("enter end range : "))
if sr>er:
    print("invalid I/P")
else:
    print("\n even numbers: ")
    for i in range(sr, er+1):
        flag=isevenodd(i)
        if flag:
            print(i, end=" ")
    print("\n odd numbers: ")
    for i in range(sr, er+1):
        flag = isevenodd(i)
        if not flag:
            print(i, end=" ")
```

WAP to print the first “n” even numbers:

```
def isevenodd(n):
    return n%2==0
count=int(input("enter a number : "))
series = 1
while count>0 :
    flag=isevenodd(series)
    if  flag:
        print(series, end=" ")
        count-=1
    series+=1
```

#25/07/2025

LEAP YEAR

Year has 2 types:

NON- CENTURY	CENTURY
EG: 2025,1098,1987,2031	EG: 2000,1700,1600,1400
Year%100! = 0 and Year%4 == 0	Year%100 == 0 and Year%400 == 0

Year=2018 ➔year(2018) %100 != 0[T] ➔year(2018) %4==0[F] ➔Not Leap Year

Year

WAP to print the year is a leap year or not

```
year = int(input("enter the year: "))
if (year%100!=0 and year%4==0)or(year%400==0):
    print(f"{year} is Leap year")
else:
    print(f"{year} is not a Leap year")
```

WAP to print the year is leap year or not using customized function

```
def year(n):
    if(n%100!=0 and n%4==0) or (n%400==0):
        return True  #Leap year
    else:
        return False  #Not a Leap Year
'''def year(n):
    return n%4==0'''
n=int(input("enter a year : "))
flag= year(n)
if flag:
    print(f"{n} is a leap year")
else:
    print(f"{n} is not a leap year")
```

Or

```
def year(n):
    return((n%100!=0 and n%4==0) or (n%400==0))

'''def year(n):
    return n%4==0'''
n=int(input("enter a year : "))
flag= year(n)
if flag:
    print(f"{n} is a leap year")
else:
    print(f"{n} is not a leap year")
```

WAP to print all the leap year's of a user defined range

```
def year(n):
    return((n%100!=0 and n%4==0) or (n%400==0))
sr=int(input("enter start year: "))
er=int(input("enter end year: "))
if sr>er:
    print("INVALID I/P")
else:
    print("leap year's: ")
    for i in range(sr,er+1):
        flag=year(i)
        if flag:
            print(i, end=" ")
```

WAP to print all the leap year and not leap year separately of a user defined range

```
def year(n):
    return((n%100!=0 and n%4==0) or (n%400==0))
sr=int(input("enter start year: "))
er=int(input("enter end year: "))
if sr>er:
    print("INVALID I/P")
else:
    print("leap year's: ")
    for i in range(sr,er+1):
        flag=year(i)
        if flag:
            print(i, end=" ")
    print("\nNot a Leap year's: ")
    for i in range(sr,er+1):
        flag=year(i)
        if flag == False:
            print(i, end=" ")
```

#28/07/2025

WAP to count a number digits in a given number

TRACING:

N 4358, 435, 43, 4, 0 Count 0, 1, 2, 3 ➔N=4358//10 Count = 0+1 ➔N=435//10 Count = 1+1 ➔N=43//10 Count = 2+1	➔N=4//10 Count = 3+1 N==0: (exit loop) Print(Count)

```
n=int(input("enter a num: "))
if n<0:
    n*=-1
count=0
while n>0:
    n=n//10
    count+=1
print(count)
```

WAP to count a number digits in a given number using Customized function

```
def numcount(n):
    if n<0:
        n*=-1
    count=0
    while n>0:
        n=n//10
        count=count+1
    return count

n=int(input("enter a num: "))
res=numcount(n)
print(f"The count of digit {n} is:",res)
```

WAP to print the count the digits of each individual number in a user defined range

```
def numcount(n):
    if n<0:
        n*=-1
    count=0
    while n>0:
        n=n//10
        count=count+1
    return count

sr=int(input("enter a start num: "))
er=int(input("enter a end num : "))
if sr<er:
    print("INVALID I/P")
else:
    for i in range(sr, er+1):
        res=numcount(i)
        print(f"the count of digit {i} is :", res)
```

[NOTE:

a) To remove a digit from a given number → Num//10
b) Whenever we need to carry forward the current updated variable value to the next cycle for further operation, use the same variable on RHS for operation and LHS for updation
c) To convert negative to positive ➔Num * -1]

ARM Strong Numbers

➔If the sum of exponent value(base→extracted digit, power→count of digit) is same as the original number then it is said to be a ARM Strong Number

TRACING:

N 153, 15, 1, 0 temp=[N] Pow = [3] asnum(153) ASN 0,27, 152, 153 ➔base = 153%10 ASN = 0+ (3**3) N=153//10 ➔base= 15%10 ASN = 27+ (5**3) N=15//10	➔base=1%10 ASN = 152+(1**3) N=1//10 n==0: (exit loop) If ASN==temp: P(“ASN”) Else P(“NOT ASN”)

```
def asnum(n):
    count=0
    while n>0:
        n=n//10
        count+=1
    return count
n=int(input("enter a number: "))
temp=n
if n<0:
    n*=-1
pow=asnum(n)
asn=0
while n>0:
    #extraxt the digit from n
    base = n%10
    #sum up the expo val
    asn+=(base**pow)
    #remove the digit
    n=n//10
if temp < 0:
    asn*=-1
if temp == asn:
    print(f"{temp} is an ASN")
else:
    print(f"{temp} is NOT an ASN")
```

WAP to check whether the given number is an ASN or not using Customized function

```
def asnum(n):
    count=0
    while n>0:
        n=n//10
        count+=1
    return count

def isasn(n):
    temp=n
    if n<0:
        n*=-1
    asn=0
    pow=asnum(n)
    while n>0:
    #extraxt the digit from n
        base = n%10
    #sum up the expo val
        asn+=(base**pow)
    #remove the digit
        n=n//10
    if temp < 0:
        asn*=-1
    return temp == asn

n=int(input("enter a number: "))
flag=isasn(n)
if flag:
    print(f"{n} is an ASN ")
else:
    print(f"{n} is not an ASN ")
```

WAP to print all the ASN numbers for an user defined range

```
def isasn(n):
    temp=n
    if n<0:
        n*=-1
    asn=0
    pow=asnum(n)
    while n>0:
    #extraxt the digit from n
        base = n%10
    #sum up the expo val
        asn+=(base**pow)
    #remove the digit
        n=n//10
    if temp < 0:
        asn*=-1
    return temp == asn

sr=int(input("enter a start num: "))
er=int(input("enter a end range: "))
if sr>er:
    print("INVALID I/P")
else:
    print("ASN NUMBERS :")
    for i in range(sr,er+1):
        flag=isasn(i)
        if flag:
            print(i)
    print("\n NOT AN ASN :")
    for i in range(sr, er+1):
        flag = isasn(i)
        if not flag:
            print(i)
```

WAP to print the first “n” ARM Strong numbers:

```
def asnum(n):
    count=0
    while n>0:
        n=n//10
        count+=1
    return count

def isasn(n):
    temp=n
    if n<0:
        n*=-1
    asn=0
    pow=asnum(n)
    while n>0:
    #extraxt the digit from n
        base = n%10
    #sum up the expo val
        asn+=(base**pow)
    #remove the digit
        n=n//10
    if temp < 0:
        asn*=-1
    return temp == asn

n=int(input("enter a number: "))
series = 1
while n>0 :
    flag=isasn(series)
    if  flag:
        print(series, end=" ")
        n-=1
    series+=1
```

#30/07/2025

WAP to reverse the number without using inbuilt function

TRACING :

N 5823, 582, 58, 5 Rev 0, 3, 32, 328 ➔5823 rem = 5823%10 Rev =(0*10)+rem N= 5823//10 ➔582 Rem= 582%10 Rev= (3*10)+rem N=582//10	➔58 Rem=58%10 Rev=(32*10)+rem N=58//10 ➔5 Rem=5%10 Rev=(328*10)+rem N=5//10 N==0: (exit loop) Print(Rev)

```
n=int(input("enter a number: "))
temp=n
if n<0:
    n*=-1
rev=0
while n>0:
    rem= n%10
    rev= (rev*10)+rem
    n=n//10
if temp<0:
    rev*=-1

print(f"the reverse order of {temp} is ", rev)
```

WAP to reverse a number using the customized function

```
def reverse(n):
    temp=n
    if n<0:
        n*=-1
    rev=0
    while n>0:
        rem= n%10
        rev= (rev*10)+rem
        n=n//10
    if temp<0:
     rev*=-1
    return rev

n=int(input("enter a number: "))
flag=reverse(n)
if flag:
    print(f"the reverse order of {n}  is", flag)
```

WAP to print all reverse number for an user defined range

```
def reverse(n):
    temp=n
    if n<0:
        n*=-1
    rev=0
    while n>0:
        rem= n%10
        rev= (rev*10)+rem
        n=n//10
    if temp<0:
     rev*=-1
    return rev

sr=int(input("enter a start num: "))
er=int(input("enter a end number: "))
if sr>er:
    print("INVALID I/P")

for i in range(sr, er+1):
    k=reverse(i)
    print(k)
```

INTEGER PALINDROME

➔if the reversal of a number is same as the original number then it is said to be an integer palindrome

WAP to print all the palindromic and non palindromic numbers separately in a user defined range

```
def intpalindrom(n):
    temp=n
    if n<0:
        n*=-1
    rev=0
    while n>0:
        rem=n%10
        rev=(rev*10)+rem
        n=n//10
    if temp<0:
        rev*=-1
    return temp==rev

sr=int(input("enter a start range: "))
er=int(input("enter a end range: "))
if sr>er:
    print("INVALID I/P")
print("PALINDROMIC NO: ")
for i in range(sr, er+1):
    flag=intpalindrom(i)
    if flag:
        print(i)

print("NON-PALINDROMIC NO: ")
for i in range(sr, er+1):
    flag=intpalindrom(i)
    if not flag:
        print(i)
```

WAP to print first n palindromic numbers

```
def intpalindrom(n):
    temp=n
    if n<0:
        n*=-1
    rev =0
    while n>0:
        rem=n%10
        rev = (rev*10)+rem
        n=n//10
    if temp<0:
        rev*=-1
    return temp==rev

n=int(input("enter a n palindromes no: "))
series=1
while n>0:
    flag=intpalindrom(series)
    if flag:
        print(series, end=" ")
        n-=1
    series+=1
```

WAP to pint first n non-palindromic numbers

```
def intpalindrom(n):
    temp=n
    if n<0:
        n*=-1
    rev=0
    while n>0:
        rem=n%10
        rev=(rev*10)+rem
        n=n//10
    if temp<0:
        rev*=-1
    return temp==rev

n=int(input("enter a number: "))
series=1
print("non palindromic no: ")
while n>0:
    flag=intpalindrom(series)
    if not flag:
        print (series, end=" ")
        n-=1
    series+=1
```

#31/07/2025

FACTORS

- A value is said to be a factor of a number only if the value can completely divide and reduce the number to zero.
- All the factors of a number will be within the range of 1 to the number itself
- Every number will have minimum of 2 numbers ➔ one and the number itself

WAP to print all the factors of given number :

[NOTE: factors, divisibility or deduction ➔ %(modules)]

TRACING

N 6 I 1, 2 ➔6%1==0[T] P(1) ➔6%2==0[T] P(2)	➔6%3==0[T] P(3) ➔6%4==0[F] ➔6%5==0[F] ➔6%6==0[T] P(6)

```
n=int(input("enter a number: "))
print(f"the factors of {n} is : ")
for i in range(1,n+1):
    if n%i==0:
        print(i, end=" ")

print("\n====================================")

def fact(n):
    for i in range(1,n+1):
        if n%i==0:
            print(i)

n=int(input("enter a number: "))
fact(n)
```

WAP to print all the factors of each number in a given user defined range :
(TASK STILL WANT TO DO)

WAP to count the number of factors for the given I/P number :

TRACING:

N 4 CF 0, 1, 2, 3 I 1, 2, 3, 4, 5 If 4%1==0:[T] Cf= 0+1	If 4%2==0:[T] Cf= 1+1 If 4%3==0:[F] If 4%4==0:[T] Cf= 2+1 Return CF

```
def fact(n):

    for i in range(1,n+1):

        if n%i==0:
            print(i, end=" ")


def countfact(n):
    countfact=0
    for i in range(1, (n+1)):
        if n%i==0:
            countfact+=1
    return countfact

n=int(input("enter a number: "))
print("the factors of ",n," is : ")
k=fact(n)
res=countfact(n)

print(f"\n the count of factors of {n} is:{res}" )
```

WAP to count the cycles of number of factors for the given input numbers

```
def fact(n):
    countcycle=0
    for i in range(1,n+1):
        countcycle+=1
        if n%i==0:
            print(i, end=" ")
    return countcycle


def countfact(n):
    countfact=0
    for i in range(1, (n+1)):
        if n%i==0:
            countfact+=1
    return countfact

n=int(input("enter a number: "))
print("the factors of ",n," is : ")
k=fact(n)
res=countfact(n)
print(f"\n the cycle of {n} is:{ k}")
print(f"\n the count of factors of {n} is:{res}" )
```

PROBLEM: The number of cycle executed to print minimal number of factor is high. Therefore the above logic is not efficient

#01/08/2025

Optimize the logic for printing the factors

➔All the factors of a number can be printed within the direct square root or lower nearest square root of a given number

TRACING:

N 24 I 1 I*I<=24:➔[T] LOGIC: I * (N//I)= N ➔1* (24//1)=24 ➔1,24	➔2*(24//2)=24 ➔2,12 ➔3*(24//3)=24 ➔3,8 ➔4*(24//4)= 24 ➔4,6 ➔5*5<=24 [F] (exit)

TRACING:

N 54 I 1 I*I<=54:➔[T] LOGIC: I * (N//I)= N ➔1* (54//1)=24 [T] ➔1,54	➔2*(54//2)=24 [T] ➔2,27 ➔3*(54//3)=24[T] ➔3,16 ➔4*(54//4)= 24 [F] ➔5*(54//5)= 24 [F] ➔6*(54//6)= 24 [T] ➔6,9 ➔7*(54//7)= 24 [F] ➔8*8<=54 [F] (exit)

WAP to print the factorial of number using nearest square roots

```
n=int(input("enter a number: "))
i=1
while i*i<=n:
    if n%i==0:
        print(i, end=" ")
        if i != (n//i):
            print((n//i), end=" ")
    i+=1
```

WAP to print the factorial of a number using customized function

```
def fact(n):
    i=1
    while i*i<=n:
        if n%i==0:
            print(i, end=" ")
            if i != (n//i):
                print((n//i), end=" ")
        i+=1

n=int(input("enter a number: "))
fact(n)
```

WAP to count the number of factors of given number

```
def fact(n):
    i=1
    while i*i<=n:
        if n%i==0:
            print(i, end=" ")
            if i != (n//i):
                print((n//i), end=" ")
        i+=1

def countfact(n):
    countfact=0
    i=1
    while i*i<=n:
        if n%i==0:
            countfact+=1
            if i!=(n//i):
                countfact+=1
        i+=1
    return countfact

n=int(input("enter a number: "))
res=countfact(n)
fact(n)
print(f"\n the count factors of {n} is : {res} ")
```

WAP to count the cycles of factors of given number

```
def fact(n):
    i=1
    countcycle=0
    while i*i<=n:
        countcycle+=1
        if n%i==0:
            print(i, end=" ")
            if i != (n//i):
                print((n//i), end=" ")
        i+=1
    return countcycle
def countfact(n):
    countfact=0
    i=1
    while i*i<=n:
        if n%i==0:
            countfact+=1
            if i!=(n//i):
                countfact+=1
        i+=1
    return countfact

n=int(input("enter a number: "))
res=countfact(n)
k=fact(n)
print(f"\n the count factors of {n} is : {res} ")
print(f"\n the count factors of {n} is : {k} ")
```

WAP to check whether the given number is prime or not

- There are no negative prime numbers
- Zero and one are not prime numbers
- The numbers which has 2 factors are called as prime numbers

TRACING :

N=7 CF 0,1,2 I 1,2,3 ➔1*1<=7 : [T] If 7%1==0➔ [T] Cf+=1 If 1!=(7//1): Cf+=1 I+=1	➔2*2<=7: [T] If 7%2==0 ➔[F] I+=1 ➔3*3<=7 : [F] [EXIT] Return CF==2 ➔[T]

```
n=int(input("enter a number: "))
i=1
countfact=0
while i*i<=n:
    if n%i==0:
        countfact+=1
        if i != (n//i):
            countfact+=1
    i+=1
if countfact==2:
    print(f"\n {n} is a prime number")
else:
    print(f"\n {n} is not a prime number")
```

WAP to check whether the given number is prime or not using customized function

```
def countfact(n):
    countfact=0
    i=1
    while i*i<=n:
        if n%i==0:
            countfact+=1
            if i!=(n//i):
                countfact+=1
        i+=1
    return countfact==2

n=int(input("enter a number: "))
flag=countfact(n)
if flag:
    print(f"\nthe {n} is prime number")
else:
    print(f"\n the {n} is not a prime number")
```

WAP to print all the prime and non prime numbers of a user defined range

```
def prime(n):
    count = 0
    i=1

    while i*i<=n:
        if n % i == 0:
            count += 1
            if i != (n//i):
                count += 1
        i+=1

    return count == 2

sr = int(input("Enter a start range: "))
er = int(input("Enter an end range: "))

if sr>er:
    print("INVALID I\P")

else:

    print(f"\nThe prime numbers from {sr} to {er} are:")
    for i in range(sr, er + 1):
        flag = prime(i)
        if flag:
            print(i, end=" ")

    print(f"\nThe non-prime numbers from {sr} to {er} are:")
    for i in range(sr, er + 1):
        flag = prime(i)
        if not flag:
            print(i, end=" ")
```

WAP to print first ‘n’ prime numbers

```python
def prime(n):
    count = 0
    i=1

    while i*i<=n:
        if n % i == 0:
            count += 1
            if i != (n//i):
                count += 1
        i+=1

    return count == 2

n=int(input("enter a num: "))
i=1

while n>0:
    flag=prime(i)
    if flag:
        print(i, end=" ")
        n-=1
    i+=1
```

WAP to print first ‘n’ non- prime number

```python
def prime(n):
    count = 0
    i=1

    while i*i<=n:
        if n % i == 0:
            count += 1
            if i != (n//i):
                count += 1
        i+=1

    return count == 2

n=int(input("enter a num: "))
i=1

while n>0:
    flag=prime(i)
    if not flag:
        print(i, end=" ")
        n-=1
    i+=1
```

#02/08/2025

GCD or HCF of the given 2 numbers

WAP to display the HCF of the given 2 numbers
(GCD→Greatest common divisor
HCF→Highest common factor
The largest common factor that divides the given 2 I/P’s Num’s)

[NOTE: The smaller number will never have a factor in the higher range of the larger number]

N1=8 N2=15 HCF=1 ➔I=2 8%2==0 and 15%2==0 : [F] ➔ I=3 8%3==0 : [F] ➔I=4 8%4==0 and 15%4==0 : [F]	➔I=5 8%5==0 : [F] ➔I=6 8%6==0 : [F] ➔ I=7 8%7==0 : [F] ➔I=8 8%8==0 and 15%8==0 : [F]

N1=12 N2=3

HCF=1, 3

➔ I=2
12%2==0 and 3%2==0 : [F]

➔I=3
12%3==0 and 3%3==0 : [T] → HCF=I

```
n1=int(input("enter a 1st number: "))
n2=int(input("enter a 2nd number: "))
lower=n1
if n2<n1:
    lower=n2
hcf=1
for i in range(2, lower+1):
    if n1%i==0 and n2%i==0:
        hcf=i
print(f"\n The highest factor of {n1} and {n2} is {hcf}")
```

WAP to display the HCF of the given 2 numbers using customized function

```
def gcd(n1,n2):
    lower=n1
    if n2<n1:
        lower=n2
    hcf=1
    for i in range(2, lower+1):
        if n1%i==0 and n2%i==0:
            hcf=i
    return hcf

n1=int(input("enter a 1st number: "))
n2=int(input("enter a 2nd number: "))
res=gcd(n1,n2)
print(f"\n The highest factor of {n1} and {n2} is {res}")
```

FIBONACCI SERIES:

- The initial 2 values of the series will always be 0 and 1
- The basic logic ➔ sum of previous 2 position values

WAP to print the Fibonacci series

Pos
6,5,4,3,2,1,0
N1
0,1,1,2,3,5,8
N2
1,1,2,3,5,8,13
Temp
1,2,3,5,8,13

➔Pos=6
Print(n1) #0
Temp=n1+n2
N1=n2
N2=temp
Pos -=1

➔Pos=5
Print(n1) #1
Temp=n1+n2
N1=n2
N2=temp
Pos -=1

➔Pos=4
Print(n1) #1
Temp=n1+n2
N1=n2
N2=temp
Pos -=1

➔Pos=3
Print(n1) #2
Temp=n1+n2
N1=n2
N2=temp
Pos -=1

➔Pos=2
Print(n1) #3
Temp=n1+n2
N1=n2
N2=temp
Pos -=1

➔Pos=1
Print(n1) #5
Temp=n1+n2
N1=n2
N2=temp
Pos -=1

Pos>0 : [F]
➔EXIT

WAP to print the Fibonacci series using customized function

```python
def fib(pos):
    n1,n2=0,1
    while pos>0:
        print(n1, end=" ")
        temp=n1+n2
        n1=n2
        n2=temp
        pos-=1

pos=int(input('enter a num: '))
fib(pos)
```

```python
def fib(pos):
    n1,n2=0,1
    for i in range(pos):
        print(n1, end=" ")
        temp=n1+n2
        n1=n2
        n2=temp
        pos+=1

pos=int(input('enter a num: '))
fib(pos)
```

#04/08/2025

STACK ➔ It is a temporary memory, where current executing block of code along with its variable gets loaded for execution

Python compiler

- It checks for syntatical mistakes
- It includes the compulsory lines of code required by PVM to execute the logic
- It converts the HLL to intermediate code / Byte code

Return

➔It is a pre-defined keyword

➔It is the last executable line of code in a function

Uses

- It helps to return value's from called function back to function call
- It returns the execution of PVM from called function back to function call
- It returns the memory that was provided during function call to PVM

Recursion ➔ The function calling itself repeatedly for certain number of times to execute the same logic

Syntax : without return value

```
Def fun_name(parameters):
        #base condition
        #logic
        Fun_name(arguments)  #recursive fun call
#initial fun call
Fun_name(initial value of parameters)
```

Syntax : with return value

```
Def fun_name(parameters):
        #base condition
        #logic
        Return Fun_name(arguments)  #recursive fun call
#initial fun call
Var_name=Fun_name(initial value of parameters)
```

#05/07/2025

[NOTE: Stack are in the book]

Function Call : PVM will allocate some memory in stack for "function and its local variables" to have space and program a logic

WAP to print 1-4 using recursion

```python
def printnum(n):
    if n<=0:
        return
    print(n, end=" ")
    printnum(n-1)

n=int(input("enter a num: "))
printnum(n)
```

```python
def printnum(n):
    if n<=0:
        return
    printnum(n-1)
    print(n , end=" ")

n=int(input("enter a num: "))
printnum(n)
```

WAP to print the Following O/P

I/P: n=5

O/P: 5 4 3 2 1 1 2 3 4 5

```python
def printnum(n):
    if n<=0:
        return
    print(n, end=" ")
    printnum(n-1)
    print(n, end=" ")

n=int(input("enter a num: "))
printnum(n)
```

I/P : n=5

O/P: 1 2 3 4 5 5 4 3 2 1

```
def num(i,n):
    if i>n:
        return
    print(i,end=" ")
    num((i+1),n)
    print(i, end=" ")

n=int(input("enter a num: "))
num(1,n)
```

Difference B/W recursion function call and initial function call

Recursion Function Call	Normal Function Call
It will be included within the recursion function definition	It will be included outside function to initiate the function execution
It passes the updated values that are required by the call	It passes the first value of parameters

#06/08/2025

WAP to count the number of digits using recursion

Recursive Logic

Logic: n=n//10

count+=1

Parameters: n, count

Base condition : if n<=0:

return count

```
def countdig(n,count):

    if n<=0:
        return count

        n=n//10
        count+=1
        return countdig(n,count)


n=int(input("enter a num: "))
res=countdig(n,0)
print(f"\n the count of digit {n} is {res}")
```

WAP to check the given number is ASN or not using recursion

Recursive logic

Logic: base=n%10

asn+=(base**pow)

n=n//10

Parameters: n, pow, asn, temp

Base condition: if n<=0:

Return temp==asn

```
def countdig(n,count):
    if n<=0:
        return count

    n=n//10
    count+=1
    return countdig(n,count)

def isasn(n,pow,asn,temp):
    if n<=0:
        return temp==asn
    base=n%10
    asn+=(base**pow)
    n//=10
    return isasn(n,pow,asn,temp)

n=int(input("enter a num: "))

pow=countdig(n,0)
flag=isasn(n,pow,0,n)
if flag:
    print(f"\n {n} is ASN")
else:
    print(f"\n{n} is not ASN")
```

WAP to reverse a number using recursion

Recursive logic :

Logic : rem=n%10

Rev=(rev*10)+rem

N=n//10

Parameters : n, rev

Base condition : if n<=0:

Return rev

```
def revnum(n,rev):

    if n==0:
        return rev

    rem=n%10
    rev=(rev*10)+rem
    n=n//10
    return revnum(n,rev)

n=int(input("enter a num: "))
res=revnum(n,0)
print(f"\n the reverse of {n} is {res}")
```

WAP to print the palindrome of a number using recursion

Recursive logic :

Logic: rem=n%10

Rev=(rev*10)+rem

N=n//10

Parameters : n, rev, temp

Base condition : if n<=0:

Return temp==rev

```
def palin(n,rev,temp):

    if n<=0:
        return temp==rev

    rem=n%10
    rev=(rev*10)+rem
    n=n//10

    return palin(n,rev,temp)

n=int(input("enter a num: "))
flag=palin(n,0,n)
if flag:
    print(f"\n {n} is palindrome")
else:
    print(f"\n {n} is not palindrome")
```

#08/08/2025

WAP to print the factors of a number using recursion

Recursive logic

Logic: n % I ==0:

Print(I, end=" ")

Parameters : n, (i+1)

Base condition : I >n

```
def fact(n,i):

    if i>n:
        return

    if n%i==0:
        print(i, end=" ")
    fact(n,(i+1))

n=int(input("enter a number: "))
fact(n,1)
```

WAP to count the factors of a number using recursion

Recursive logic

Logic: n % I ==0:

Count+=1

Parameters : n, (i+1), count

Base condition : I>n

```
def countfact(n,i,count):

    if i>n:
        return count

    if n%i==0:
        count+=1

    return countfact(n,(i+1),count)

n=int(input("enter a number: "))
res=countfact(n,1,0)
print(res)
```

WAP to print all the factors of a number using optimized logic :

Recursive Logic :

Logic : if n % i==0:

P(i)

If I != (n//i):

P(n//i)

Parameters: n, i+1

Base condition : I*I>n

```
def fact(n,i):

    if i*i>n:
        return
    if n%i==0:
        print(i, end=" ")
        if i != (n//i):
            print((n//i), end=" ")
    fact(n,(i+1))

n=int(input("enter a num: "))
print(f"\nthe fact of {n} is: ")
fact(n,1)
```

WAP to count all the factors of a number using optimized logic :

Recursive Logic :

Logic : if n % 1==0:

Count+=1

If I !=(n//i):

Count+=1

Parameters: n, i+1,count

Base condition : I*I>n

```
def countfact(n,i,count):

    if i*i>n:
        return count

    if n%i==0:
        count+=1
        if i!=(n//i):
            count+=1
    return countfact(n,i+1,count)

n=int(input("enter a num: "))
res=countfact(n,1,0)
print(f"\nthe count of fact {n} is {res}")
```

WAP to check whether the given number is prime or not :

Recursive Logic :

Logic : if n%1==0:

Count+=1

If i!=n//i:

Count+=1

Parameters : n,i+1,count

Base condition : i*i>n:

Return count==2

```
def prime(n,i,count):
    if i*i>n:
        return count == 2

    if n%i==0:
        count+=1
        if i != (n//i):
            count+=1

    return prime(n,(i+1),count)

n=int(input("enter a num: "))
flag=prime(n,1,0)
if flag:
    print(f"\n {n} is prime number")
else:
    print(f"\n {n} is not prime")
```

#12/08/2025

WAP to print the GCD of given two numbers using recursion

Recursive logic

Logic : if n1 % I == 0 and n2 % I == 0

Hcf = i

Parameters : n1,n2,I, hcf , lower

Base condition : if i>lower:

Return hcf

```
def gcd(n1,n2,i,hcf,lower):
    if i>lower:
        return hcf
    if n1%i==0 and n2%i==0:
        hcf=i
    return gcd(n1,n2,i+1,hcf,lower)

n1=int(input("enter a num1: "))
n2=int(input("enter a num2: "))
lower=n1
if n2<n1:
    lower=n2

res= gcd(n1,n2,1,1,lower)
print(res)
```

Efficient logic for finding the GCD of n1 and n2 :

➔**Euclidian's algm**:

N1 and n2

Gcd(n1,n2)= Gcd((n1-n2), n2)

Where, n1>n2

Reduce it till n1 == 0 then ➔ n2 is the GCD

N1 = 18 N2 = 16 N2>N1: →F Findgcd(18,16)= findgcd((18-16), 16) Findgcd(16,2)= findgcd((16-2), 2) Findgcd(14,2)= findgcd((14-2), 2) Findgcd(12, 2)=findgcd((12-2), 2) Findgcd(10,2)= findgcd((10-2),2)	Findgcd(8,2)= findgcd((8-2),2) Findgcd(6,2)= findgcd((6-2),2) Findgcd(4,2)= findgcd((4-2),2) Findgcd(2,2)= findgcd((2-2),2) If n1==0: Return n2

```
def gcd(n1,n2):

    if n1==0:
        return n2

    if n1<n2:
        n1,n2=n2,n1
    return gcd((n1-n2),n2)


n1=int(input("enter a num: "))
n2=int(input("enter a num: "))

res= gcd(n1,n2)
print(res)
```

Using another method with same Euclidian algm :

N1=18
N2=16

N2>n1: →F
Findgcd(18,16)=findgcd((18%16),16)

Findgcd(16,2)=findgcd((16%2),2)

N1==0:
Return n2

```python
def gcd(n1,n2):

    if n1==0:
        return n2

    if n1<n2:
        n1,n2=n2,n1
    return gcd((n1%n2),n2)


n1=int(input("enter a num: "))
n2=int(input("enter a num: "))

res= gcd(n1,n2)
print(res)
```

[NOTE : 2147483648 → it is the largest number in integer range

-2147483648 → it is the lowest number in integer range]

General programming approach:

1. Good logic → Brute force approach
2. Better logic → Better approach
3. Best logic → Efficient logic (optimized)

WAP to print the Fibonacci series using recursion:

Recursive logic

Logic : print(n)
pos-1, n1=(n2), n2=(n1+n2)

Base condition : if pos <=0:
Return

Tracing :

→Pos= 6, n1=0, n2=1 Print(n1) #0 →pos= 5, n1=1, n2=1 Print(n1) #1 →pos= 4, n1=1, n2=2 Print(n1) #1 →pos= 3, n1=2, n2=3 Print(n1) #2	→pos=2, n1=3, n2=5 Print(n1) #3 →pos=1, n1=5, n2=8 Print(n1) #5 →pos=0, n1=8, n2=13 Pos==0:→T return

```python
def fib(pos,n1,n2):
    if pos<=0:
        return
    print(n1, end=" ")

    fib(pos-1,n2,n1+n2)

pos=int(input("enter a num: "))
fib(pos,0,1)
```

#13/08/2025

WAP to print the nth fibonacii number using recursion
Recursive logic
Logic: fib(n-1)+ fib(n-2)
Base condition: if n==0 or n==1:
Return n

```python
def fib(n):

    if n==0 or n==1:
        return n

    return fib(n-1)+ fib(n-2)

n=int(input("enter a number: "))
res= fib(n)
print(f"\n the fibonacii of {n} is {res}")
```

[NOTE : **why recursion** ?
➔It helps to solve a problem by reducing a complex expression to its simplest form]

WAP to print the factorial of the given number
Recursive logic
Logic : n * fact(n-1)
Parameters : n
Base condition : if n==1 or n==0:
Return 1

```python
def fact(n):
    if n==1 or n==0:
        return 1

    return n * fact(n-1)

n=int(input("enter a num: "))
res=fact(n)
print(res)
```

#14/08/2025

Happy numbers

➔If the number is reduced to 1, then it is called a happy number, or if a number is reduced to 4, then it is called unhappy number

➔Any other number should be summed up with the squares of extracted digit in a number

TRACING

N=32	N=18
➔32==1: →F 32==4: →F (3)^2+(2)^2= 9+4= 13 ➔13==1: →F 13==4: →F (1)^2+(3)^2= 1+9= 10 ➔10==1: →F 10==4: →F (1)^2+(0)^2= 1+0= 1 ➔1==1: →T Return True ➔32 is a happy number	➔18==1: →F 18==4: →F (1)^2+(8)^2= 1+64= 65 ➔65==1: →F 65==4: →F (6)^2+(5)^2= 36+25= 61 ➔61==1: →F 61==4: →F (6)^2+(1)^2= 36+1= 37 ➔37==1: →F 37==4:→F (3)^2+(7)^2= 9+49= 58 ➔58==1: →F 58==4:→F (5)^2+(8)^2= 25+64= 89 ➔89==1: →F 89==4: →F (8)^2+(9)^2= 64+81= 145 ➔145==1: →F 145==4: →F (1)^2+(4)^2+(5)^2= 1+16+25= 42 ➔42==1: →F 42==4: →F (4)^2+(2)^2= 16+4= 20 ➔20==1: →F 20==4: →F (2)^2+(0)^2= 4+0= 4 ➔4==1: →F 4==4: →T Return false ➔18 is a unhappy number

[**NOTE** : to repeat a repeated logic

1. Nested loop
2. Loop inside the recursive cycle]

WAP to check whether the given number is happy or unhappy number

Recursion logic

Logic: sum=0

While n>0:

Base=n%10

Sum=sum+(base**2)

N=n//10

Parameters : n

Base condition : if n==1:

Return True

If n==4:

Return False

```
def happy(n):
    if n==1:
        return True
    elif n==4:
        return False
    sum=0
    while n>0:
        base=n%10
        sum+=(base**2)
        n=n//10
    return happy(sum)
n=int(input("enter a number: "))
print(happy(n))
```

#16/08/2025

[Note :

1. Print ()➔ it is pre-define function, it helps to take the control to the beginning of the next line, it also helps to print the argument passed on the O/P screen during run time and then take the control to the next line
2. End ➔ it is a pre-defined attribute that keeps back the control at the and of the same line, it can hold a string in it]

PATTERNS

➔Combination of rows and columns

-------- ➔ sleeping/ horizontal / rows

|| ➔standing/ vertical/ columns

➔user I/P(n) ➔the total number of rows to be printer in the pattern

➔varieties: star, hollow, number, alphabetic

➔Types :

i. Non-combinational:➔direct “n” value to be considered
ii. Combinational :➔(2 * n) value to be considered

Non-Combinational

Combinational

[NOTE : General range of n value is 4<=n<=15]

NON-Combinational Pattern

Triangles :

[NOTE : I➔It is dependent on “n”

J➔it is dependent on “n” or “I” or “third variable”

a) LHS right angled Triangle

Relationship Table

I	J
1	1-1
2	1-2
3	1-3
4	1-4

Tracing

```
n=int(input("enter a num: "))
for i in range(1,n+1):
    for j in range(1,i+1):
        print("*", end=" ")
    print()
```

Square Pattern

```
n=int(input("enter a num: "))
for i in range(1,n+1):
    for j in range(1,n+1):
        print("*", end=" ")
    print()
```

#18/08/2025

Inverted LHS Right angle triangle

```
n=int(input("enter a num: "))
for i in range(n,(1-1),-1):
    for j in range(1,(i+1)):
        print("*", end="")
    print()
```

I	J
4	1<=4
3	1<=3
2	1<=2
1	1<=1

RHS right angle triangle

```
n=int(input("enter a num: "))
for i in range(1,(n+1)):
    for k in range(n, i, -1):
        print(" ", end="")
    for j in range(1,(i+1)):
        print("*", end="")
    print()
```

I	K(explicit spaces)	J
1	4-2 (till 1)	1-1
2	4-3 (till 2)	1-2
3	4-4 (till 3)	1-3
4	0 (till 4)	1-4

Pyramid pattern

[NOTE : In straight triangle the order of I and j is same ,

In inverted triangle the order of I and j will be opposite

Comparison table

RHS Right angle triangle Pyramid

I	No of explicit spaces	No of stars	No of explicit spaces	No of stars
1	3	1	3	1
2	2	2	2	2
3	1	3	1	3
4	0	4	0	4

```
n=int(input("enter a num: "))
for i in range(1,(n+1)):
    for k in range(n, i, -1):
        print(" ", end="")
    for j in range(1,(i+1)):
        print("*", end=" ")
    print()
```

Inverted RHS Right angle triangle

```python
n=int(input("enter a num : "))
for i in range(n,(1-1),-1):
    for k in range(n, i, -1):
        print(" ", end="")
    for j in range(1, (i+1)):
        print("*", end="")
    print()
```

I	K(explicit spaces)	J
4	0 (till 4)	1-4
3	4-4(till 3)	1-3
2	4-3 (till 2)	1-2
1	4-2 (till 1)	1-1

Inverted Pyramid pattern

RHS Inverted Right angle Triangle

Inverted Pyramid

I	No of explicit spaces	No of stars	No of explicit spaces	No of stars
4	0	4	0	4
3	1	3	1	3
2	2	2	2	2
1	3	1	3	1

```python
n=int(input("enter a num : "))
for i in range(n,(1-1),-1):
    for k in range(n, i, -1):
        print(" ", end="")
    for j in range(1, (i+1)):
        print("*", end=" ")
    print()
```

#19/08/2025

ODD Number Right angle Triangle

I	J	Odd[Variable]
1	1-1	1
2	1-3	3
3	1-5	5
4	1-7	7

```python
n=int(input("enter a num: "))
odd=1
for i in range(1,n+1):
    for j in range(1, (odd+1)):
        print("* ", end="")
    print()
    odd+=2
```

Odd number pyramid

```python
n=int(input("enter a num: "))
odd=1
for i in range(1,n+1):
    for k in range(n,i,-1):
        print(" ", end="")
    for j in range(1, (odd+1)):
        print("*", end="")
    print()
    odd+=2
```

```
enter a num: 5
    *
   ***
  *****
 *******
*********
```

Odd number inverted pyramid

```
n=int(input("enter a num: "))
odd=n*2-1
for i in range(n,1-1,-1):
    for k in range(n,i,-1):
        print(" ", end="")
    for j in range(1, odd+1):
        print("*", end="")
    print()
    odd-=2
```

```
enter a num: 5
*********
 *******
  *****
   ***
    *
```

Parallelogram

```
n=int(input("enter a num: "))
for i in range(1,n+1):
    for k in range(n,i,-1):
        print(" ", end="")
    for j in range(1, n+1):
        print("*", end="")
    print()
```

```
enter a num: 5
    *****
   *****
  *****
 *****
*****
```

```
n=int(input("enter a num : "))
for i in range(n,1-1,-1):
    for k in range(n, i, -1):
        print(" ", end="")
    for j in range(1, (n+1)):
        print("*", end="")
    print()
```

```
enter a num : 5
*****
 *****
  *****
   *****
    *****
```

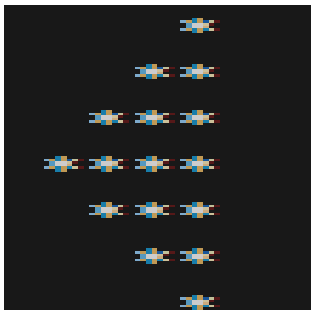
Combinational Patterns

```
*
**
***
****
***
**
*
```

I	j	Noc
1	1-1	1
2	1-2	2
3	1-3	3
4	1-4	4
5	1-3	3
6	1-2	2
7	1-1	1

```
n=int(input("enter a num: "))
noc=1
for i in range(1,n*2):
    for j in range(1,(noc+1)):
        print("*", end="")
    print()
    if i<n:
        noc+=1
    else:
        noc-=1
```

Inverted Combinational Patterns



I	j	Noc	K
1	1-1	1	4-2[Till 1]
2	1-2	2	4-3[Till 2]
3	1-3	3	4-4[Till 3]
4	1-4	4	0
5	1-3	3	4-4[Till 3]
6	1-2	2	4-3[Till 2]
7	1-1	1	4-2[Till 1]

```
n=int(input("enter a num: "))
noc=1
for i in range(1,n*2):
    for k in range(n,noc,-1):
        print(" ", end="")
    for j in range(1,(noc+1)):
        print("*", end="")
    print()
    if i<n:
        noc+=1
    else:
        noc-=1
```

Diamond Patterns

Inverted Pattern

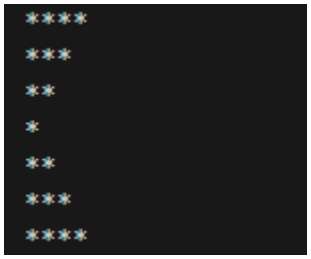
Diamond Pattern

I	No of Explicit spaces	No of stars	No of Explicit Spaces	No of stars
1	3	1	3	1
2	2	2	2	2
3	1	3	1	3
4	0	4	0	4
5	1	3	1	3
6	2	2	2	2
7	3	1	3	1

```
n=int(input("enter a num: "))
noc=1
for i in range(1,n*2):
    for k in range(n,noc,-1):
        print(" ", end="")
    for j in range(1,(noc+1)):
        print("* ", end="")
    print()
    if i<n:
        noc+=1
    else:
        noc-=1
```

```
enter a num: 4
   *
  * *
 * * *
* * * *
 * * *
  * *
   *
```

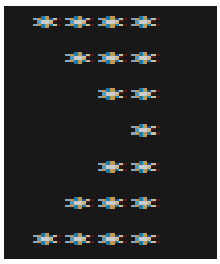
Combinational Pattern



I	j	Noc
1	1-4	4
2	1-3	3
3	1-2	2
4	1-1	1
5	1-2	2
6	1-3	3
7	1-4	4

```
n=int(input("enter a num : "))
noc=n
for i in range(1,(n*2)):
    for j in range(1, (noc+1)):
        print("*", end="")
    print()
    if i<n:
        noc-=1
    else:
        noc+=1
```

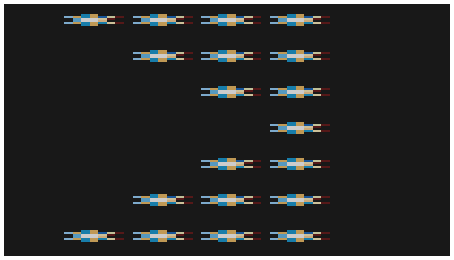
Combinational Pattern

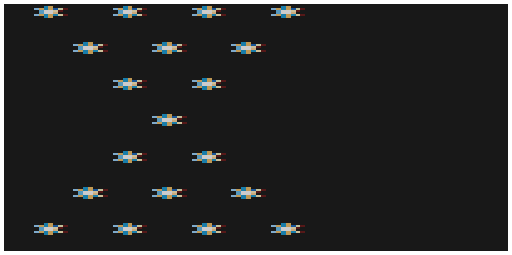


I	j	Noc	K
1	1-4	4	0
2	1-3	3	4-4[Till 3]
3	1-2	2	4-3[Till 2]
4	1-1	1	4-2[Till 1]
5	1-2	2	4-3[Till 2]
6	1-3	3	4-4[Till 3]
7	1-4	4	0

```
n=int(input("enter a num : "))
noc=n
for i in range(1,(n*2)):
    for k in range(n,noc, -1):
        print(" ", end="")
    for j in range(1, (noc+1)):
        print("*", end="")
    print()
    if i<n:
        noc-=1
    else:
        noc+=1
```

Combinational Pattern





I	No of Explicit spaces	No of stars	No of Explicit Spaces	No of stars
1	0	4	0	4
2	1	3	1	3
3	2	2	2	2
4	3	1	3	1
5	2	2	2	2
6	1	3	1	3
7	0	4	0	4

```
n=int(input("enter a num : "))
noc=n
for i in range(1,(n*2)):
    for k in range(n,noc, -1):
        print(" ", end="")
    for j in range(1, (noc+1)):
        print("* ", end="")
    print()
    if i<n:
        noc-=1
    else:
        noc+=1
```

#20/08/2025

Butterfly pattern

I J(1-7)

Rows	LHS(1-4)	RHS(4-7)	NOC	NOR
1	1-1	7-7	1	7
2	1-2	6-7	2	6
3	1-3	5-7	3	5
4	1-4	4-7	4	4
5	1-3	5-7	3	5
6	1-2	6-7	2	6

7	1-1	7-7	1	7

Derivation of the condition

If J <= NOC or j>=NOR

Print(“*”, end=””)

Else:

Print(“ ”, end=””)

```
n=int(input("enetr a num: "))
noc=1
nor=n*2-1
for i in range(1, n*2):
    for j in range(1, n*2):
        if j<=noc or j>=nor:
            print("*", end="")
        else:
            print(" ", end="")
    print()
    if i<n:
        noc+=1
        nor-=1
    if i>=n:
        noc-=1
        nor+=1
```

```
*       *
**     **
***   ***
********
***   ***
**     **
*       *
```

[NOTE :

➔whenever the space is within the pattern, then include a logic or condition within the “J” loop such that, if the condition is satisfied the print(character) else print(space)

➔if the pattern is printing any 2 characters then include an “if-else” statement within “j” loop]

Hollow Patterns

```
n=int(input("enter a num: "))
for i in range(1, n+1):
    for j in range(1, n+1):
        if i==1 or i==n or j==1 or j==n:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
```

```
enter a num: 5
* * * * *
*       *
*       *
*       *
* * * * *
```

```python
n=int(input("enter a num: "))
if n%2==0:
    n+=1
for i in range(1, n+1):
    for j in range(1, n+1):
        if (i==1 or i==n or j==1 or j==n or i==j or i+j==n+1):
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
```

```
enter a num: 5
* * * * *
* *   * *
*   *   *
* *   * *
* * * * *
```

```python
n=int(input("enter a num:"))
for i in range(n,1-1,-1):
    for j in range(1,n+1):
        if i==j or j==1 or i==n:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

```
enter a num:5
*****
*  *
* *
**
*
```

```python
n=int(input("enter a num:"))
for i in range(n,1-1,-1):
    for k in range(n,i,-1):
        print(" ", end="")
    for j in range(1,n+1):
        if i==j or j==1 or i==n:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

```
enter a num:5
*****
 *  *
  * *
   **
    *
```

```python
n=int(input("enter a num:"))
for i in range(1,(n+1)):
    for k in range(n,i,-1):
        print(" ", end="")
    for j in range(1,n+1):
        if i==j or j==1 or i==n:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

```
enter a num:5
    *
   **
  * *
 *  *
*****
```

```python
n=int(input("enter a num:"))
for i in range(1,(n+1)):
    for j in range(1,n+1):
        if i==j or j==1 or i==n:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

```
enter a num:5
*
**
* *
*  *
*****
```

```
n=int(input("enter a num: "))
if n%2==0:
    n+=1
for i in range(1, n+1):
    for j in range(1, n+1):
        if (j==1 or j==n  or i==j or i+j==n+1):
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
```

```
enter a num: 5
*       *
* *   * *
*   *   *
* *   * *
*       *
```

```
n=int(input("enter a num:"))
noc=n
for i in range(1,n*2):
    for k in range(n,noc,-1):
        print(" ",end="")
    for j in range(1,(noc+1)):
        if noc==j or j==1  or i==1 or n==noc:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
    if i<n:
        noc-=1
    else:
        noc+=1
```

```
enter a num:5
* * * * *
 *     *
  *   *
   * *
    *
   * *
  *   *
 *     *
* * * * *
```

```
n=int(input("enter a num:"))
noc=n
for i in range(1,n*2):
    for k in range(n,noc,-1):
        print(" ",end="")
    for j in range(1,(noc+1)):
        if noc==j or j==1  or i==1 or i==n*2-1:
            print("*", end="")
        else:
            print(" ", end="")
    print()
    if i<n:
        noc-=1
    else:
        noc+=1
```

```
enter a num:5
*****
 *   *
  * *
   **
    *
   **
  * *
 *   *
*****
```

[NOTE : Table are all in Notes]

```python
n=int(input("enter a num:"))
noc=n
for i in range(1,n*2):
    for j in range(1,(noc+1)):
        if noc==j or j==1  or i==1 or i==n*2-1:
            print("*", end="")
        else:
            print(" ", end="")
    print()
    if i<n:
        noc-=1
    else:
        noc+=1
```

```
enter a num:5
*****
*   *
* *
**
*
**
* *
*   *
*****
```

```python
n=int(input("enter a num:"))
noc=1
for i in range(1,n*2):
    for k in range(n,noc,-1):
        print(" ",end="")
    for j in range(1,(noc+1)):
        if noc==j or j==1:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
    if i<n:
        noc+=1
    else:
        noc-=1
```

```
enter a num:5
    *
   * *
  *   *
 *     *
*       *
 *     *
  *   *
   * *
    *
```

```python
n=int(input("enter a num:"))
noc=1
for i in range(1,n*2):
    for k in range(n,noc,-1):
        print(" ",end="")
    for j in range(1,(noc+1)):
        if noc==j or j==1:
            print("*", end="")
        else:
            print(" ", end="")
    print()
    if i<n:
        noc+=1
    else:
        noc-=1
```

```
enter a num:5
    *
   **
  * *
 *  *
*   *
 *  *
  * *
   **
    *
```

```python
n=int(input("enter a num:"))
noc=1
for i in range(1,(n*2)):
    for j in range(1,(noc+1)):
        if noc==j or j==1:
            print("*", end="")
        else:
            print(" ", end="")
    print()
    if i<n:
        noc+=1
    else:
        noc-=1
```

```
enter a num:5
*
**
* *
*  *
*   *
*  *
* *
**
*
```

#22/08/2025

Numerical Pattern

- At every updated row, if it the complete row is printing the same value. Then the value present in the variable I
- At every updated row, if it is starting from same point and moving till certain point or vice versa. Then it is printing the value present in variable J
- If the pattern is filled with continuous series, then it is printing the value from a third variable

```
n=int(input("enter a num: "))
for i in range(1,n+1):
    for j in range(1,n+1):
        print(i, end=" ")
    print()
```

```
enter a num: 4
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4
```

```
n=int(input("enter a num: "))
for i in range(n,1-1,-1):
    for j in  range(n,i-1,-1):
        print(i, end=" ")
    print()
```

```
enter a num: 4
4
3 3
2 2 2
1 1 1 1
```

```
n=int(input("enter a num: "))
for i in range(1,n+1):
    for j in  range(n,i-1,-1):
        print(j, end=" ")
    print()
```

```
enter a num: 4
4 3 2 1
4 3 2
4 3
4
```

```
n=int(input("enter a num: "))
for i in range(n,0,-1):
    for j in  range(1,n+1):
        print(i, end=" ")
    print()
```

```
enter a num: 4
4 4 4 4
3 3 3 3
2 2 2 2
1 1 1 1
```

```
n=int(input("enter a num: "))
for i in range(1,n+1):
    for j in range(i,n+1):
        print(j,end=" ")
    print()
```

```
enter a num: 4
1 2 3 4
2 3 4
3 4
4
```

```
n=int(input("enter a num: "))
for i in range(1,n+1):
    for k in range(1,i+1):
        print(" ",end="")
    for j in range(i,n+1):
        print(j,end="")
    print()
```

```
enter a num: 4
 1234
  234
   34
    4
```

```
n=int(input("enter a num: "))
for i in range(1,n+1):
    for j in range(i,n+i):
        print(j, end="")
    print()
```

```
enter a num: 4
1234
2345
3456
4567
```

```
n=int(input("enter a num: "))
noc=n
for i in range(1,n*2):
    for j in range(1,noc+1):
        print(j, end=" ")
    print()
    if i<n:
        noc-=1
    else:
        noc+=1
```

```
enter a num: 4
1 2 3 4
1 2 3
1 2
1
1 2
1 2 3
1 2 3 4
```

```
n=int(input("enter a num: "))
noc=n
for i in range(1,n*2):
    for j in range(noc,0,-1):
        print(j, end="")
    print()
    if i<n:
        noc-=1
    else:
        noc+=1
```

```
enter a num: 4
4321
321
21
1
21
321
4321
```

#25/08/2025

Number Pattern

```
n=int(input("enter a num:"))
for i in range(1,n+1):
    for j in range(1,n+1):
        if i<j:
            print(i, end="")
        else:
            print(j, end="")
    print()
```

```
enter a num:4
1111
1222
1233
1234
```

```python
n=int(input("enter a num:"))
for i in range(n,0,-1):
    for j in range(n,(1-1),-1):
            if i>j:
                print(j, end="")
            else:
                print(i, end="")
    print()
```

```
enter a num:4
4321
3321
2221
1111
```

```python
n=int(input("enter a num:"))
for i in range(1,n+1):
    for j in range(1,n+1):
            if i<j:
                print(j, end="")
            else:
                print(i, end="")
    print()
```

```
enter a num:4
1234
2234
3334
4444
```

```python
n=int(input("enter a num:"))
for i in range(n,0,-1):
    for j in range(n,(1-1),-1):
            if i>j:
                print(i, end="")
            else:
                print(j, end="")
    print()
```

```
enter a num:4
4444
4333
4322
4321
```

```python
n=int(input("enter a num:"))
for i in range(1,n+1):
    for j in range(1,i+1):
            if (i+j)%2==0:
                print(1, end="")
            else:
                print(0, end="")
    print()
```

```
enter a num:4
1
01
101
0101
```

```python
n=int(input("enter a num:"))
for i in range(1,n+1):
    for j in range(1,n+1):
            if (i+j)%2==0:
                print(0, end="")
            else:
                print(1, end="")
    print()
```

```
enter a num:4
0101
1010
0101
1010
```

```python
n=int(input("enter a num:"))
noc=1
for i in range(1,n+1):
    for j in range(1,n+1):
        print(noc, end=" ")
        noc+=1
    print()
```

```
enter a num:4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
```

```python
n=int(input("enter a num: "))
noc=1
nor=n*2
for i in range(1,n+1):
    for j in range(1,n+1):
        if i%2!=0:
            print(noc, end=" ")
            countdecre=noc
        else:
            print(countdecre, end=" ")
            countdecre-=1
        noc+=1
    print()
    countdecre+=n
```

```
enter a num: 5
1 2 3 4 5
10 9 8 7 6
11 12 13 14 15
20 19 18 17 16
21 22 23 24 25
```

```python
n=int(input("enter a num: "))
noc=1
for i in range(1,n+1):
    for j in range(1,i+1):
        if i%2!=0:
            print(noc, end=" ")
            countdecre=noc
        else:
            print(countdecre, end=" ")
            countdecre-=1
        noc+=1
    print()
    countdecre+=i+1
```

```
enter a num: 4
1
3 2
4 5 6
10 9 8 7
```

#27/08/2025

Alphabetic Patterns

ASCII Value's ➔ The integer representation of each individual character maintained by virtual machine

➔Totally 256 ascii value's are available ranging from 0-255

➔Extended version of ASCII value's are called as UNICODE values

" "[Space] →32

"0"- "9" → 48-57

"A"- "Z" →65-90

"a" – "z" →97-122

2- Inbuilt functions to preform manipulations on characters:

1. Ordinal Function:

   Syntax: Ord(single character)

   →Returns the integer representation of the provided character

2. Character Function:

   Syntax: chr(whole Num)

   →Returns the character representation of the provided number

Logic

- Capital letters ➔ (64+val)
- Small letters ➔(96+val)

```
n=int(input("enter a num: "))
for i in range(1,n+1):
    for j in range(1,i+1):
        print(chr(96+i), end="")
    print()
```

```
enter a num: 5
a
bb
ccc
dddd
eeeee
```

```
n=int(input("enter a num: "))
for i in range(n,1-1,-1):
    for j in range(1,i+1):
        print(chr(64+j), end="")
    print()
```

```
enter a num: 5
ABCDE
ABCD
ABC
AB
A
```

```python
n=int(input("enter a num: "))
for i in range(1,n+1):
    for j in range(1,n+1):
        if i%2==0:
            print(chr(96+j), end="")
        else:
            print(chr(64+j), end="")
    print()
```

```
enter a num: 5
ABCDE
abcde
ABCDE
abcde
ABCDE
```

```python
n=int(input("enter a num: "))
for i in range(n,1-1,-1):
    for j in range(1,i+1):
        if j%2==0:
            print(chr(64+j), end="")
        else:
            print(chr(96+j), end="")
    print()
```

```
enter a num: 5
aBcDe
aBcD
aBc
aB
a
```

```python
n=int(input("enter a num: "))
for i in range(1,n+1):
    for j in range(1,n+1):
        if (i+j)%2==0:
            print(chr(64+j), end=" ")
        else:
            print(chr(96+j), end=" ")
    print()
```

```
enter a num: 5
A b C d E
a B c D e
A b C d E
a B c D e
A b C d E
```

```python
n=int(input("enter a num: "))
noc=1
for i in range(1,n+1):
    for j in range(1,n+1):
        if noc%2!=0:
            print(chr(64+noc), end=" ")
        else:
            print(chr(96+noc), end=" ")
        noc+=1
    print()
```

```
enter a num: 5
A b C d E
f G h I j
K l M n O
p Q r S t
U v W x Y
```

```python
n=int(input("enter a num: "))
noc=1
for i in range(1,n*2):
    for j in range(noc,n+1):
            print(chr(96+j), end=" ")

    print()
    if i<n:
        noc+=1
    else:
        noc-=1
```

```
enter a num: 5
a b c d e
b c d e
c d e
d e
e
d e
c d e
b c d e
a b c d e
```

#28/08/2025

Data Structure → we can store multiple individual elements into the same shared memory location in a particular organized (organized or unorganized) manner

e.g.: arrays, linked list, stack, queue, tress, graphs

ARRAYS/LISTS

➔It is the data structure that can hold multiple individual elements of same or different data types into the same shared memory location

➔It stores elements into continuous memory locations

➔one shared memory location is further sub-divided and each sub division has it's own whole number called as index, to access each element individually

➔Its stores data's of homogeneous(same data type) or heterogeneous(different data type)

➔Arrays are represented ➔ "[]", where elements are segregated with ","

➔It can hold duplicates

➔It is a indexed data structure

Index

- It is a whole number that starts from 0 to positive integer
- First index → 0, last index → Len(array)-1
- Index is assigned by PVM during execution time
- Manually, if we try to access an Index that is not available in the created list then PVM will throw an “Index Error”

Len()

- It is pre-defined function which returns the count of number of elements available in a created in a iterable object
- Syntax :

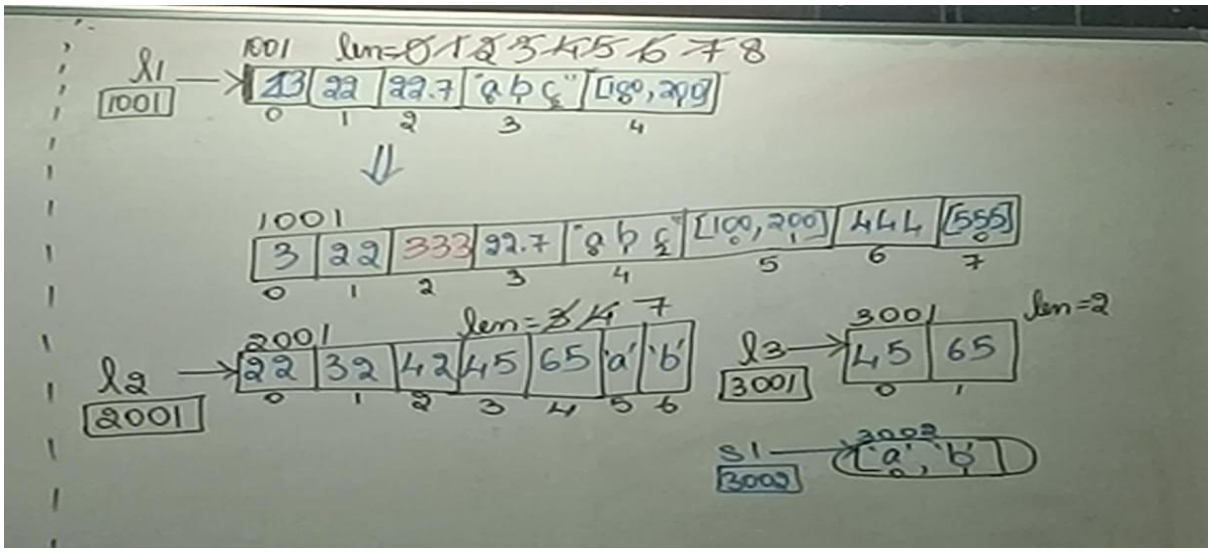
  Len(created_iterable_object)

```
l1=[1,3,"abc",23.5]
print(len(l1))
l1=[]
print(len(l1))
t1=(1,2,3,6,72,2,3)
print(len(t1))
s1="eflaewkjhfrliawuhdn;oqi2u3p892u3wn3diul"
print(len(s1))
set1={2,3,4,2,3,4,1,1}
print(len(set1))
dict={101:"ramu", 102:"sita"}
print(len(dict))  #return the count of num of itesm
print(len(10))  #error bcz single valued data is not allowed
```

Arrays

```
l1=[]
# l1[0]=100  #INDEX ERROR
l1.append(1)
l1.append(22)
l1.append(22.7)
l1.append("abc")
l1.append([100,200])
print(l1)
l1[0]=3    #replacement of my existing value
print(l1)
l1.insert(2,333)
print(l1)
l1.insert(99,444)
print(l1)
l1.insert(100,[555])
print(l1)
l1.insert(88,234)
print(l1)
l2=[22,32,42]
print(l2)
l3=[45,65]
print(l3)
l2.extend(l3)
print(l2)
s1="ab"
print(s1)
l2.extend(s1)
print(l2)
#l2.extend(301)
#print(l2)      #error
```

```
[1, 22, 22.7, 'abc', [100, 200]]
[3, 22, 22.7, 'abc', [100, 200]]
[3, 22, 333, 22.7, 'abc', [100, 200]]
[3, 22, 333, 22.7, 'abc', [100, 200], 444]
[3, 22, 333, 22.7, 'abc', [100, 200], 444, [555]]
[3, 22, 333, 22.7, 'abc', [100, 200], 444, [555], 234]
[22, 32, 42]
[45, 65]
[22, 32, 42, 45, 65]
ab
[22, 32, 42, 45, 65, 'a', 'b']
```



#29/07/2025

WAP to create dynamic integer array with user defined values

```
def createarray():
    l1=[]
    while True:
        try:
            n=int(input("enter a value: "))
            l1.append(n)
        except Exception as e:
            return l1


arr=createarray()
print(arr)
```

```
enter a value: 3
enter a value: 4
enter a value: 5
enter a value:
[3, 4, 5]
```

WAP to create a dynamic character array by inserting user defined values

```
def stringarray():
    l1=[]
    while True:
            n=input("enter a value: ")
            if n=="":
                return l1
            l1.append(n)


arr=stringarray()
print(arr)
```

```
enter a value: a
enter a value: d
enter a value: v
enter a value: e
enter a value: f
enter a value: g
enter a value:
['a', 'd', 'v', 'e', 'f', 'g']
```

```
def chararray():
    l1=[]
    while True:
        n=input("enter a char: ")
        if n=="":
            return l1
        l1.append(n[0])

arr=chararray()
print(arr)
```

```
enter a char: dfsjer
enter a char: fwkjlehf
enter a char: eflkjwehfi
enter a char:
['d', 'f', 'e']
```

Linear Search

WAP to implement linear search on a given array / WAP to search for the target element in a given array

Expected O/P:

1. Boolean value
2. Index at which the target is found

#01/09/2025

WAP to find the largest element in a given array along with its Index

```
def createarray():
    l1=[]
    while True:
        try:
            n=int(input("enter a num: "))
            l1.append(n)
        except Exception as e:
            return l1

def findmaximum(nums):
    max=(-2**31)-1
    maxind=-1
    for i in range(0,len(nums)):
        if max < nums[i]:
            max=nums[i]
            maxind=i
    return max,maxind

print("enter a values into the array")
nums=createarray()
res, resind=findmaximum(nums)
print(f"\n the maximum element is: {res}, its index is: {resind}")
```

```
enter a values into the array
enter a num: 2
enter a num: 3
enter a num: 4
enter a num: 5
enter a num: 6
enter a num: 7
enter a num:

 the maximum element is: 7, its index is: 5
```

WAP to find the smallest element in a given array along with its Index

```
def createarray():
    l1=[]
    while True:
        try:
            n=int(input("enter a num: "))
            l1.append(n)
        except Exception as e:
            return l1

def findminimum(nums):
    max=(2**31)
    maxind=-1
    for i in range(0, len(nums)):
        if max > nums[i]:
            max=nums[i]
            maxind=i
    return max, maxind

print("enter the array values: ")
nums=createarray()
res, resind= findminimum(nums)
print(f"\n the minimum element is : {res}, index is : {resind}")
```

```
enter the array values:
enter a num: 3
enter a num: 4
enter a num: 5
enter a num: 6
enter a num: 1
enter a num:

 the minimum element is : 1, index is : 4
```

WAP to fin the secmax element along with it's index in a given array

```
def createarray():
    l1=[]
    while True:
        try:
            n=int(input("enter a num: "))
            l1.append(n)
        except Exception as e:
            return l1
def secmax(nums):
    max=(-2**31)-1
    maxind=-1
    secmax=(-2**31)-1
    secmaxind=-1
    for i in range(0, len(nums)):
        if max< nums[i]:
            secmax=max
            secmaxind=maxind
            max=nums[i]
            maxind=i
        elif max!=nums[i]:
            if secmax< nums[i]:
                secmax= nums[i]
                secmaxind=i
    return max, maxind, secmax, secmaxind

print("enter the array values: ")
nums=createarray()
res, resind, secres, secresind= secmax(nums)
print(f"\n the first largest values is: {res} at its index is: {resind}")
print(f"\n the second largest values is : {secres} at its index is : {secresind}")
```

```
enter the array values:
enter a num: 100
enter a num: 29
enter a num: 380
enter a num: 379
enter a num: -234
enter a num:

 the first largest values is: 380 at its index is: 2

 the second largest values is : 379 at its index is : 3
```

WAP to fin the secmin element along with it's index in a given array

```
def createarray():
    l1=[]
    while True:
        try:
            n=int(input("enter a num: "))
            l1.append(n)
        except Exception as e:
            return l1
def secmin(nums):
    min=(2**31)-1
    minind=-1
    secmin=(-2**31)-1
    secminind=-1
    for i in range(0, len(nums)):
        if min> nums[i]:
            secmin=min
            secminind=minind
            min=nums[i]
            minind=i
        elif min!=nums[i]:
            if secmin> nums[i]:
                secmin= nums[i]
                secminind=i
    return min, minind, secmin, secminind

print("enter the array values: ")
nums=createarray()
res, resind, secres, secresind= secmin(nums)
print(f"\n the first minimum values is: {res} at its index is: {resind}")
print(f"\n the second minimum values is : {secres} at its index is : {secresind}")
```

```
enter the array values:
enter a num: 2
enter a num: 3
enter a num: -89
enter a num: 298
enter a num: 21
enter a num: -2
enter a num: 3
enter a num:

 the first minimum values is: -89 at its index is: 2

 the second minimum values is : -2 at its index is : 5
```

[NOTE : if we more than 2 return values in a function , we can store it in a list, and should remember the index values of that return elements]

#03/09/2025

SELECTION SORT

For Ascending

```python
def createarray():
    l1=[]
    while True:
        try:
            n=int(input("enter a num: "))
            l1.append(n)
        except Exception as e:
            return l1

def selectionsort(arr):
    n=len(arr)
    for i in range(0, n-1):
        actualind=(n-1)-i
        currmax, currmaxind= -2**31, -1
        for j in range(0, n-i):
            if currmax < arr[j]:
                currmax= arr[j]
                currmaxind= j
        arr[actualind], arr[currmaxind]= (arr[currmaxind], arr[actualind])


arr=createarray()
print("oringinal arr:", arr)
selectionsort(arr)
print(f"\n the sorted array: ", arr)
```

```
enter a num: 9
enter a num: 8
enter a num: 7
enter a num: 54
enter a num: 3
enter a num: 4
enter a num: 7
enter a num: 8
enter a num:
oringinal arr: [9, 8, 7, 54, 3, 4, 7, 8]

 the sorted array:  [3, 4, 7, 7, 8, 8, 9, 54]
```

For Descending

```python
def createarray():
    l1=[]
    while True:
        try:
            n=int(input("enter a num: "))
            l1.append(n)
        except Exception as e:
            return l1

def selectionsort(arr):
    n=len(arr)
    for i in range(0, n-1):
        actualind= (n-1)-i
        currmin, currminind= 2**31, -1
        for j in range(0, n-i):
            if currmin > arr[j]:
                currmin=arr[j]
                currminind= j
        arr[actualind], arr[currminind]= arr[currminind], arr[actualind]

arr=createarray()
print("oringinal arr:", arr)
selectionsort(arr)
print(f"\n the sorted array: ", arr)
```

```
 the sorted array:  [3, 4, 7, 7, 8, 8, 9, 54]
enter a num: 2
enter a num: 3
enter a num: 4
enter a num: 5
enter a num: 6
enter a num: 7
enter a num: -1
enter a num:
oringinal arr: [2, 3, 4, 5, 6, 7, -1]

 the sorted array:  [7, 6, 5, 4, 3, 2, -1]
```

#04/09/2025

Merging of array

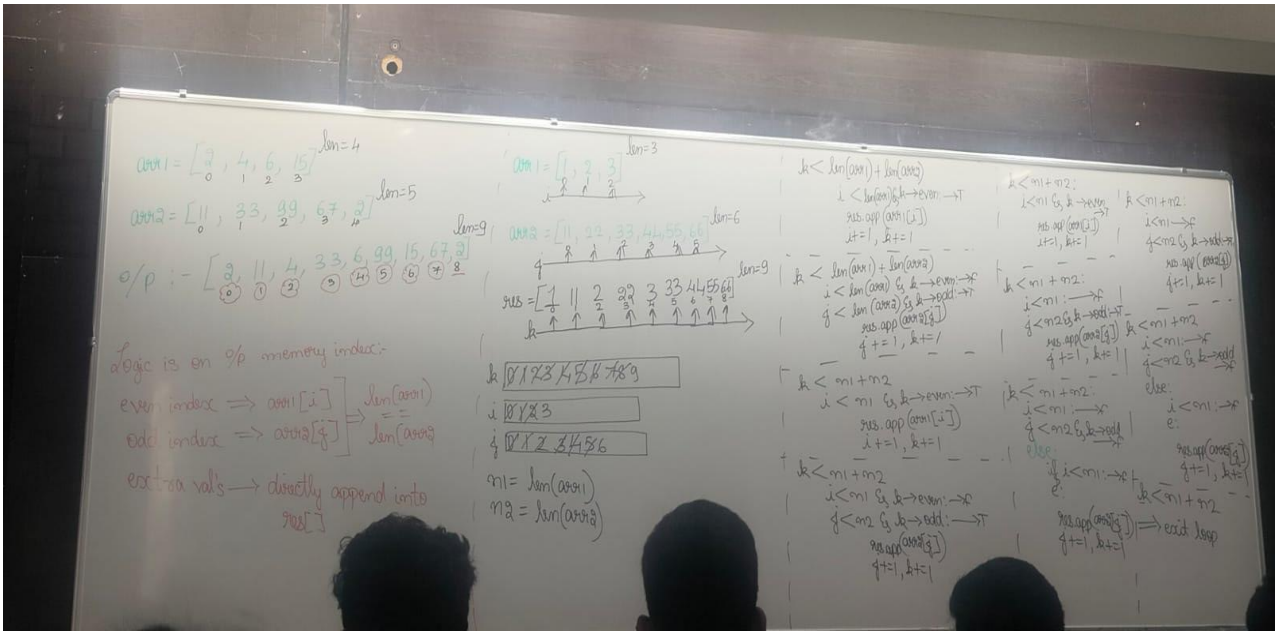
I/P : Arr1=[11,22,33,44]

Arr2=[1,3,2,4,4,5]

Excepted O/P : [1,22,33,44,1,3,2,4,4,5]

1. '+' ➔ Concatenation
   - Advantages : Not corrupting the I/P array
   - Disadvantages : A 3rd memory is being used
2. Extend ()
   - Advantages : No need of 3rd memory
   - Disadvantages : Corrupting the I/P array
3. Append()
   - Advantages : No need of 3rd memory
   - Disadvantages : Corrupting the I/P array
4. Insert()
   - Advantages : No need of 3rd memory
   - Disadvantages : Corrupting the I/P array

WAP to merge given two array at alternate index



```python
def createarray1():
    l1=[]
    while True:
        try:
            n=int(input("enter a value for array: "))
            l1.append(n)
        except Exception as e:
            return l1

arr1=createarray1()
arr2=createarray1()

res=[]
i, j, k =0, 0, 0
n1, n2=len(arr1), len(arr2)

while k<n1+n2:
    if i<n1 and k%2==0:
        res.append(arr1[i])
        i+=1
    elif j<n2 and k%2!=0:
        res.append(arr2[j])
        j+=1
    elif i<n1:
        res.append(arr1[i])
        i+=1
    elif j<n2:
        res.append(arr2[j])
        j+=1
    k+=1

print(res)
```

```
enter a value for array: 1
enter a value for array: 2
enter a value for array: 3
enter a value for array: 4
enter a value for array: 5
enter a value for array: 6
enter a value for array:
enter a value for array: 11
enter a value for array: 22
enter a value for array: 33
enter a value for array: 44
enter a value for array:
[1, 11, 2, 22, 3, 33, 4, 44, 5, 6]
```

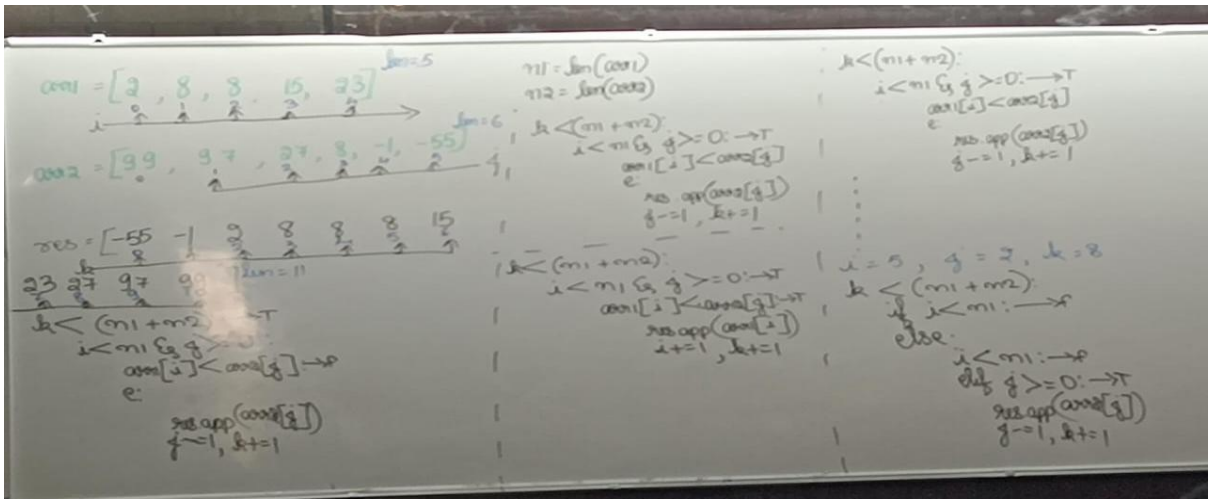
#05/09/2025

WAP to merge the given first array in ascending order and second array in descending order , merge the sored arrays in ascending sorted order

```python
def createarray():
    l1=[]
    while True:
        try:
            n=int(input("enter a value: "))
            l1.append(n)
        except Exception as e:
            return l1

def mergesort(arr1, arr2):
    n1, n2=len(arr1), len(arr2)
    i, j= 0, len(arr2)-1
    res=[]
    for k in range(0, n1+n2):
        if i<n1 and j>=0:
            if arr1[i]<arr2[j]:
                res.append(arr1[i])
                i+=1
            else:
                res.append(arr2[j])
                j-=1
        else:
            if i<n1:
                res.append(arr1[i])
                i+=1
            elif j>=0:
                res.append(arr2[j])
                j-=1
        k+=1
    return res


print("enter the values of array1:")
arr1=createarray()
print("\n enter the values of array2:")
arr2=createarray()

res=mergesort(arr1, arr2)

print("array 1:", arr1)
print("array 2:", arr2)
print("sorted array:", res)
```

```
enter the values of array1:
enter a value: 1
enter a value: 4
enter a value: 6
enter a value: 9
enter a value: 10
enter a value: 11
enter a value: 24
enter a value:

 enter the values of array2:
enter a value: 9
enter a value: 7
enter a value: 3
enter a value: 2
enter a value: -1
enter a value: -99
enter a value:
array 1: [1, 4, 6, 9, 10, 11, 24]
array 2: [9, 7, 3, 2, -1, -99]
sorted array: [-99, -1, 1, 2, 3, 4, 6, 7, 9, 9, 10, 11, 24]
```

Tracing :



WAP to print the reverse the given array

```
def createarray():
    l1=[]
    while True:
        try:
            n=int(input("enter a value: "))
            l1.append(n)
        except Exception as e:
            return l1

def reverse(arr):
    i,j=0,len(arr)-1
    while i<j:
        arr[i], arr[j]= arr[j], arr[i]
        i+=1
        j-=1

print("enter the values of array: ")
arr=createarray()
print("present arrays is:", arr)
reverse(arr)
print("reverse of arrays is:", arr)
```

```
enter the values of array:
enter a value: 1
enter a value: 2
enter a value: 3
enter a value: 5
enter a value: 7
enter a value: 89
enter a value: 92
enter a value:
present arrays is: [1, 2, 3, 5, 7, 89, 92]
reverse of arrays is: [92, 89, 7, 5, 3, 2, 1]
```

#06/09/2025

Test Cases

Eg1: Nums1=[2,8,10,18,0,0,0,0] m=4, n=4, m+n
Nums2=[1,2,2,9] n=4
O/P: nums1=[1,2,2,2,8,9,10,18]

Eg2 : Nums1=[0] m=0, n=1, m+n
Nums2=[23] n=1
O/P: nums1=[23]

Eg3 : Nums1=[18,88] m=2, n=0, m+n
Nums2=[ ] n=0

Eg4: Nums1= [1,2,3,0,0,0,0] m=3, n=4, m+n

Nums2=[2,2,2,2] n=4

- If we move the curser from LHS to RHS then there is a very huge possibility to loose the original values from Nums1 as Nums1 itself is the O/P memory

N1=[1,2,2,0,0,0,0] m=3, n=4, m+n N2=[1,2,2,2] Updated N1= [] →k=(3+4)-1 I=(3-1) J=(4-1) →if n1[i]>n2[j]: →T N1[k]= n1[i] i-=1, k-=1 →if n1[i]>n2[j]: →F Else: N1[k]=n2[j] j-=1, k-=1	→If n1[i]>n2[j]: →F Else: N1[k]=n2[j] j-=1, k-=1 →If n1[i]>n2[j]: →F Else: N1[k]=n2[j] j-=1, k-=1 →if n1[i]>n2[j]: →T N1[k]= n1[i] i-=1, k-=1 →If n1[i]>n2[j]: →F Else: N1[k]=n2[j] j-=1, k-=1

SUBARRAYS : ➔It occurs in continuous series

Arr=[1,2,3,4]

[1]

[1,2]

[1,2,3]

[1,2,3,4]

---

[2]

[2,3]

[2,3,4]

---

[3]

[3,4]

---

[4]

SUBSEQUENCE : ➔ a mini array that is formed out of the major array by picking random elements

Arr=[1,2,3,4,5]

[2,5]

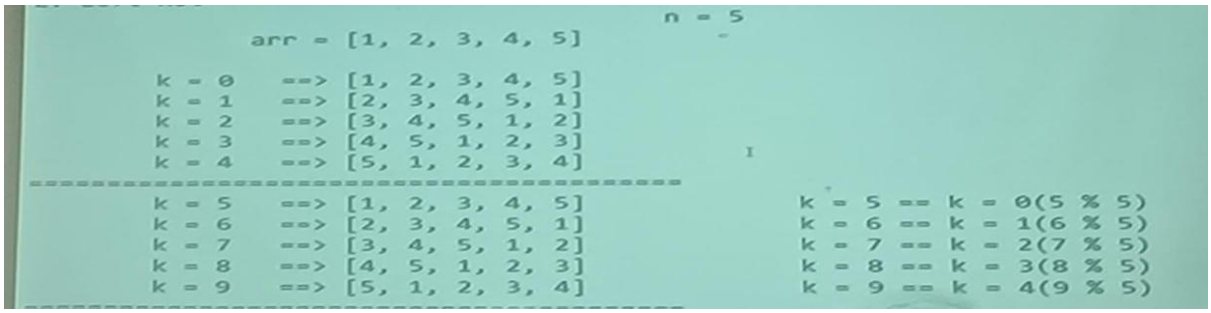
[1,2,5]

[4,1]

#08/09/2025

ROTATION OF ARRAY

1. Right Rotation




```python
def reversearray(arr,i,j):
    while i<j:
        arr[i], arr[j]= arr[j], arr[i]
        i+=1
        j-=1

def arrayrightrot(arr,k):
    n=len(arr)
    if k>n:    #to overcome indexerror
        k=k%n    #to reduce the no of rotations
    reversearray(arr,0,k-1)
    reversearray(arr,k,n-1)
    reversearray(arr,0,(n-1))

arr=[1,2,3,4,5]
print("original array", arr)
k=int(input("enter a k value: "))
arrayrightrot(arr,k)
print("rotated arrya:", arr )
```

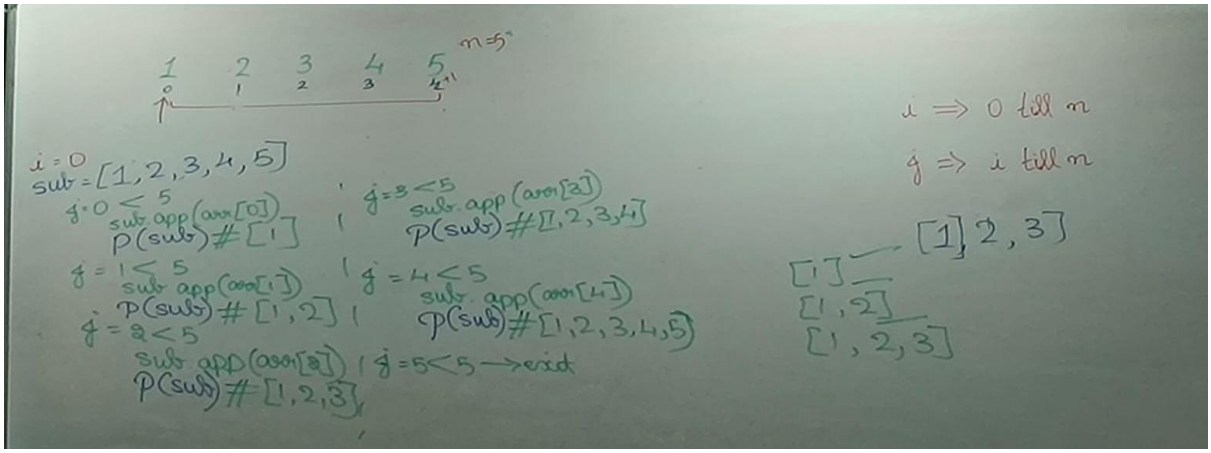
```
original array [1, 2, 3, 4, 5]
enter a k value: 6
rotated arrya: [2, 3, 4, 5, 1]
```

2. Left Rotation

```python
def reversearray(arr,i,j):
    while i<j:
        arr[i], arr[j]= arr[j], arr[i]
        i+=1
        j-=1

def arrayrightrot(arr,k):
    n=len(arr)
    if k>n:    #to overcome indexerror
        k=k%n    #to reduce the no of rotations
    reversearray(arr,0,n-1)
    reversearray(arr,k,n-1)
    reversearray(arr,0,k-1)

arr=[1,2,3,4,5]
print("original array", arr)
k=int(input("enter a k value: "))
arrayrightrot(arr,k)
print("rotated arrya:", arr )
```

```
original array [1, 2, 3, 4, 5]
enter a k value: 7
rotated arrya: [4, 5, 1, 2, 3]
```

WAP to print all the subarrays in the given array




```python
def subarray(arr):
    n=len(arr)
    for i in range(0,n):
        sub=[]
        for j in range(i,n):
            sub.append(arr[j])
            print(sub)
        print("====================================")

arr=[1,2,3,4,5]
subarray(arr)
```

```
[1]
[1, 2]
[1, 2, 3]
[1, 2, 3, 4]
[1, 2, 3, 4, 5]
====================================
[2]
[2, 3]
[2, 3, 4]
[2, 3, 4, 5]
====================================
[3]
[3, 4]
[3, 4, 5]
====================================
[4]
[4, 5]
====================================
[5]
====================================
```

#09/09/2025

WAP to return the list of subarrays of a given array

I=0
J➔0 to 4 →0,1,2,3,4
Sub=[]
K➔0 to 0 [1]
Sub=[]
K➔0 to 1 [1,2]
Sub=[]
K➔0 to 2 [1,2,3]
Sub=[]
K➔0 to 3 [1,2,3,4]
Sub=[]
K➔0 to 4 [1,2,3,4,5]
Sub=[]

➔ I=1
J➔1 to 4 →1,2,3,4
Sub=[]
K➔1 to 1 [2]
Sub=[]
K➔1 to 2 [2,3]
Sub=[]
K➔1 to 3 [2,3,4]
Sub=[]
K➔1 to 4 [2,3,4,5]
Sub=[]

➔I=2
J➔2 to 4 →2,3,4
Sub=[]
K➔2 to 2 [3]
Sub=[]
K➔2 to 3 [3,4]
Sub=[]
K➔2 to 4 [3,4,5]
Sub=[]

➔ I=3
J➔3 to 4 →3,4
Sub=[]
K➔3 to 3 [4]
Sub=[]
K➔3 to 4 [4,5]
Sub=[]

➔ I=4
J➔4 to 4 →4
Sub=[]
K➔4 to 4 [5]
Sub=[]

```python
def subarray(arr):
    n=len(arr)
    main=[]
    for i in range(0, n):
        # Start values of each subarray
        for j in range(i,n):
            #range of each subarray
            sub=[]
            # Fetch values and form subarray
            for k in range(i,j+1):
                sub.append(arr[k])
            main.append(sub)
    return main

arr=[1,2,3,4,5]
res=subarray(arr)
for i in range(0,len(res)):
    print(res[i])
```

```
[1]
[1, 2]
[1, 2, 3]
[1, 2, 3, 4]
[1, 2, 3, 4, 5]
[2]
[2, 3]
[2, 3, 4]
[2, 3, 4, 5]
[3]
[3, 4]
[3, 4, 5]
[4]
[4, 5]
[5]
```

```python
def subarray(arr):
    n=len(arr)
    main=[]
    count=0
    for i in range(0, n):
        # Start values of each subarray
        for j in range(i,n):
            #range of each subarray
            sub=[]
            # Fetch values and form subarray
            for k in range(i,j+1):
                sub.append(arr[k])
            main.append(sub)
            count+=1
    return main, count

arr=[1,2,3,4,5]
res, count=subarray(arr)
for i in range(0,len(res)):
    print(res[i])
print("total sub arrays:",count)
```

[NOTE : the Count of subarray can be done using the formula
(n* (n+1))//2, where n is length of sub array]

+ve sum

Add +ve num → value will increase
Sum=10+3=13

Add -ve num →value will decrease
Sum=10-3=7

-ve sum

Add +ve num → value will increase
Sum= -10+3=-7

Add -ve num →value will decrease
Sum= -10 -3= -13

<u>If we carry forward a -ve num:</u>

Num=[-5,-4,-1,-7,-8]

Sum = 0,-5,-9,-10,-17,-25

Max=-2**31, -5 ➔wrong answer

I=0 ➔sum=0+(-5)
I=1 ➔ sum=(-5)+(-4) = -9
I=2 ➔ sum=(-9)+(-1) = -10
I=3 ➔ sum=(-10)+(-7) = -17
I=4 ➔ sum=(-17)+(-8) = -25
➔exit loop

Re-Initialize a -ve num:

Num =[-5,-4,-1,-7,-8]

Sum= 0, -5, -, -4, 0, -1, 0, -7, 0, -8, 0
Max= -2**31, -5, -4, -1

I=0 ➔sum = 0+(-5) → sum<0 : sum=0
I=1 ➔sum = 0+(-4) → sum<0 : sum=0
I=2 ➔sum = 0+(-1) → sum<0 : sum=0
I=3 ➔sum = 0+(-7) → sum<0 : sum=0
I=4 ➔sum = 0+(-8) → sum<0 : sum=0
➔exit loop

#10/09/2025

MAX subarray

```python
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        sum=0
        max=-2**31
        for i in range(0,len(nums)):
            sum=sum+nums[i]
            if max<sum:
                max=sum
            if sum<0:
                sum=0
        return max
```

Input

nums =

[-2,1,-3,4,-1,2,1,-5,4]

Output

6

Num=[-2, 1, -3, 4, -1, 2, 1, -5, 4] Len=9 Sum=0, -2, 0, 1, -2, 0, 4, 3, 5, 6, 1, 5 Max= -2**31, -2, 1, 4, 5, 6 ➔1<9: →T Sum=0+(-2) If max<sum: →T Max= sum(-2) Is sum<0: Sum=0 ➔ 2<9: →T Sum=0+(1) If max<sum: →T Max= sum(1) Is sum<0: →F	➔ 3<9: →T Sum=1+(-3) If max<sum: →F Is sum<0: →T Sum=0 ➔…….. ➔8<9 : →T Sum=1+(4) If max<sum: →F If sum<0: →F ➔9<9: →F ➔Exit loop

WAP to print the nested subarray

```
def createarray():
    outer=[]
    while True:
        inner=[]
        while True:
            try:
                n=int(input("enter a values: "))
                inner.append(n)

            except Exception as e:
                break
        if inner==[]:
            return outer
        outer.append(inner)


arr=createarray()
print(arr)
```

```
enter a values: 1
enter a values: 2
enter a values: 3
enter a values:
enter a values: 1
enter a values: 2
enter a values:
enter a values: 1
enter a values: 2
enter a values:
enter a values:
[[1, 2, 3], [1, 2], [1, 2]]
```